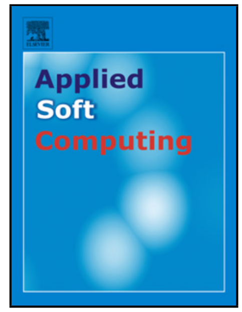
Accepted Manuscript



Title: Orthogonal PSO Algorithm for Economic Dispatch of Thermal Generating Units Under Various Power Constraints in Smart Power Grid

Authors: Loau Tawfak Al Bahrani, Jagdish Chandra Patra

PII: S1568-4946(17)30241-7
DOI: http://dx.doi.org/doi:10.1016/j.asoc.2017.04.059
Reference: ASOC 4192

To appear in: *Applied Soft Computing*

Received date: 15-11-2016
Revised date: 21-4-2017
Accepted date: 26-4-2017

Orthogonal PSO Algorithm for Economic Dispatch of Thermal Generating Units Under Various Power Constraints in Smart Power Grid

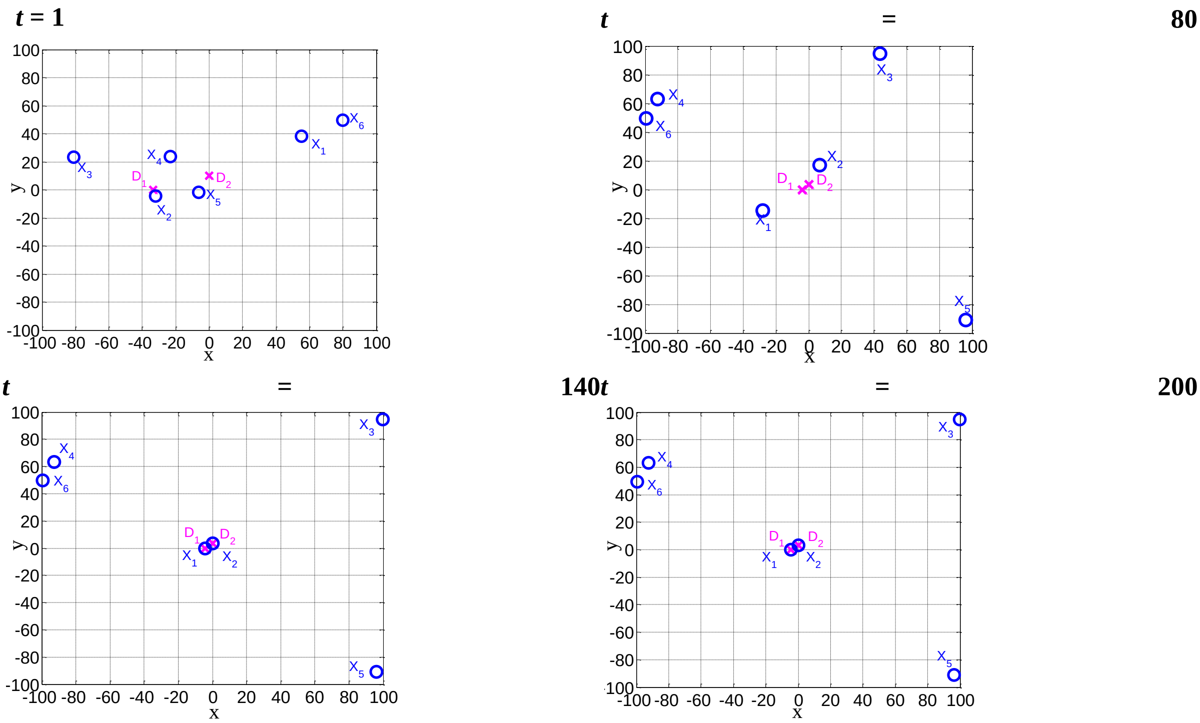
Loau Tawfak Al Bahrani, Jagdish Chandra Patra

Swinburne University of Technology, Melbourne, Australia

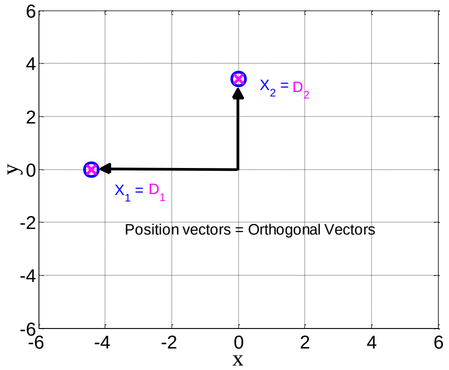
E-mail: lalbahrani@swin.edu.au; JPatra@swin.edu.au

Graphical Abstract

The OPSO algorithm is applied to obtain global optimum of a non-convex multimodal objective function.



***t* = 200 (Magnified)**



The movement of positions X_i, i = 1, 2, ...,6 of six particles based on orthogonal vectors D_1 and D_2, two active group particles (1 and 2) and four passive group particles (3, 4, 5 and 6) searching for global optimum [2.0,–3.0].

Highlights

- An orthogonal PSO (OPSO) algorithm is proposed to improve the global PSO algorithm.
- The OPSO algorithm is successfully applied to solve non-convex multimodal objective functions.
- An orthogonal diagonalization process is applied to *d* best particles in a swarm.
- The *d* orthogonal vectors are constructed in each iteration as guide to obtain global optimum.
- The OPSO algorithm is tested with three power systems and ten CEC 2015 benchmark functions.

Abstract

Particle swarm optimization (PSO) algorithm has been successfully applied to solve various optimization problems in science and engineering. One such popular one is called global PSO (GPSO) algorithm. One of major drawback of GPSO algorithm is the phenomenon of "zigzagging", that leads to premature convergence by falling into local minima. In addition, the performance of GPSO algorithm deteriorates for high-dimensional problems, especially in presence of nonlinear constraints. In this paper we propose a novel algorithm called, orthogonal PSO (OPSO) that alleviates the shortcomings of the GPSO algorithm. In OPSO algorithm, the *m* particles of the swarm are divided into two groups: active group and passive group. The *d* particles of the active group undergo an orthogonal diagonalization process and are updated in such way that their position vectors become orthogonally diagonalized. In the OPSO algorithm, the particles are updated using only one guide, thus avoiding the conflict between the two guides that occurs in the GPSO algorithm. We applied the OPSO algorithm for solving economic dispatch (ED) problem by taking three power systems under several power constraints imposed by thermal generating units (TGUs) and smart power grid (SPG), for example, ramp rate limits, and prohibited operating zones. In addition, the OPSO algorithm is also applied for ten selected shifted and rotated CEC 2015 benchmark functions. With extensive simulation studies, we have shown superior performance of OPSO algorithm over GPSO algorithm and several existing evolutional computation techniques in terms of several performance measures, e.g., minimum cost, convergence rate, consistency, and stability. In addition, using unpaired t-Test, we have shown the statistical significance of the OPSO algorithm against several contending algorithms including top-ranked CEC 2015 algorithms.

Keywords: Orthogonal particle swarm optimization; orthogonal diagonalization; economic dispatch; ramp rate limits; prohibited operating zones.

1. Introduction

Economic dispatch (ED) of power is one of the fundamental problems in smart power grid (SPG) operations. Its objective is to allocate the load demand among committed thermal generating units (TGUs) in most economical manner, while satisfying various practical power constraints imposed by the TGUs and SPG. In order to make SPG more efficient, flexible, adaptive and reliable, these constraints need to be integrated with smart meters and sensors, advanced communication technology and high-performance computing machines [1]. Therefore, there is a need to develop new computational techniques to solve ED problem that is compatible with rapid technological evolution in SPG.

Traditionally, the cost function of a TGU is approximately represented by a quadratic or a piecewise quadratic function [2-3]. However, due to several power constraints, e.g., ramp rate limits (RRLs) and prohibited operating zones (POZs), the cost function of an on-line TGU becomes non-convex and non-smooth with multiple modes, i.e., multimodal objective function. Because, the operating range of on-line TGUs is restricted by their RRLs and discontinuities in the cost curve due to their POZs.

In the past decades, many optimization techniques including traditional methods have been adopted in order to find the optimum power dispatch and the rate of optimum product for each on-line TGU. Some of the traditional methods include linear programming [4], quadratic programming [5], Lagrange relaxation [6], Lambda iterative method [7], and dynamic programming [8]. These methods offer certain advantages, for example, they only need to run once and do not have any problem specific parameters to specify. However, the traditional methods are able to solve the ED problem only when the cost function is piecewise linear and monotonically increasing.

To overcome the above mentioned deficiencies, several evolutionary computation techniques (ECTs) have been proposed to tackle the non-convex ED problems. Some of the popular ECTs include genetic algorithm (GA) [9], simulated annealing [10], artificial immune system (AIS) [11], artificial bee colony [12], evolutionary programming [13], differential evolution (DE) [14], harmony search [15]. Another popular ECT is the particle swarm optimization (PSO) algorithm proposed by Kennedy and Eberhart [16-17]. The PSO algorithm can be divided into types, one for local PSO (LPSO) and the other for global PSO (GPSO) which is more popular. Some of the advantages of application of GPSO algorithm in solving ED problem are: firstly, it imposes a few or no restrictions on the shape of the cost function; secondly, it has a few parameters and is easy to execute. However, the GPSO algorithm is suitable only for solving optimization problems that have a continuous domain and it is prone to get trapped into local minima when applied to multimodal functions.

The GPSO algorithm mimics the behaviour of swarm population of some animal species, such as, birds or fish flocking. The GPSO algorithm operates by initializing the particles (with possible solutions) of the swarm randomly and searching for global optima by updating the position and the velocity of each particle iteratively. While updating, each particle in a swarm uses its own personal experience and the best experience of the swarm as two guides through linear summation. Thus, a particle that has best experience among the personal experiences of all particles is considered to be a best solution. The reasons for poor performance of the GPSO algorithm can be summarized as follows. Firstly, the learning strategy of the GPSO algorithm depends on the fact that each particle in a swarm adjusts its search trajectory according to its own personal experience and its neighbors' experiences. Therefore, each particle in a swarm obtains two possible solutions, one from its personal experience and the other from its neighborhood's experience and then sums them together. The problem here is not only existence of the summation, but also the presence of two guides. These two guides may have a large difference or may even be opposite directions at the early search step, which may lead the particles to pull to local minima trapping and may lead to early convergence. Often, they remain in opposite directions until final search stage. Secondly, these two guides and their linear summation may cause a phenomenon called "Oscillation or zigzagging" [18-20]. This phenomenon becomes more prominent with high dimensional search space in which the particles remain in a confused state and are unable to control their search trajectories.

Several ECTs have been proposed in recent years to boost the global search ability for solving different types and forms of unimodal and multimodal objective functions. Some of these ECTs have been focused on solving non-convex multimodal cost function and improving the performance of the GPSO and LPSO algorithms in order to obtain a global optimum and to prevent the falling into local minima. For example, self-organizing hierarchical PSO (SOH-PSO) algorithm [3], modified PSO algorithm [21], PSO with modified stochastic acceleration factors (PSO-MSAF) [22], an improved PSO algorithm [23], hybrid gradient PSO (HGPSO) [24], hybrid PSO with mutation (HPSOM) algorithm [24], hybrid PSO with wavelet mutation (HPSOWM) algorithm [24], random drift PSO (RDPSO) [25], simulated annealing PSO (SA-PSO) algorithm [26], new PSO local random search (NPSO-LRS) algorithm [27], anti-predatory PSO (APSO) [28], chaotic PSO (CPSO) algorithm [29], and quantum mechanism PSO (QMPSO) algorithm [30] have been proposed. In [25], eleven ECTs, i.e., GA, DE, ant colony search algorithm (ACSA), bee colony optimization (BCO), AIS, firefly algorithm (FA), APSO, HGPSO, HPSOM, HPSOWM, RDPSO have been applied to the ED problem with three different power systems.

Other ECTs have been applied on solving other types of unimodal and multimodal objective functions, i.e., shifted and rotated CEC 2015 benchmark objective functions [31-32]. For example, local Lipschits underestimate differential evolution (LLUDE) [33], strategy adaptation differential evolution (SaDE) [33], JADE: adaptive differential evolution [33], composite differential evolution (CoDE) [33], self-adaptive binary variant differential evolution (SbaDE) [34], directionally driven self-regulating PSO (DD-SRPSO) [35], extraordinariness PSO (EPSO) [36], shrinking hypersphere PSO with gravitational search algorithm (SHPSO-GSA) [37] have been proposed.

In order to make a fair comparison, some other ECTs are also used in the literature. They include fully decentralized approach (FDA) [38], biogeography-based optimization (BBO) algorithm [39], genetic algorithm with API (GA-API) algorithm [40], mixed-integer quadratically constrained quadratic programming (MIQCQP) [41], enhanced gradient simplified swarm optimization algorithm (EGSSOA) [42], Lambda logic (λ-logic) [43], self-tuning hybrid differential evolution [44], two-phase neural network [45], synergic predator-prey optimization (SPPO) algorithm [46], and chaotic teaching-learning-based optimization algorithm [47], self optimization based adaptive DE with linear population, L-SHADE and eigenvector-based crossover and successful-parent-selecting, SPS-L-SHADE-EIG [48], DE with success-based parameter adaption (DEsPA) algorithm [49], mean-variance mapping optimization (MVMO) algorithm [50-51], and tuned covariance matrix evolution strategy (TunedCMAES) [52].

In this paper, we propose a novel algorithm called orthogonal PSO (OPSO) algorithm with a new learning strategy to solve non-convex ED problem for TGUs under several practical TGUs and power grid constraints and to improve the performance by overcoming the drawbacks of GPSO algorithm. The OPSO algorithm consists of a swarm with m particles that looks for optimal solution in a d-dimensional search space ($m \geq d$). The swarm population is divided into two groups: an active group of best personal experience of d particles and another passive group of personal experience of remaining ($m - d$) particles. The position vectors associated with the m particles undergo an orthogonal diagonalization (OD) process in which the d orthogonal guidance vectors in the active group are obtained. In each iteration, using only one guide, the velocity and position vectors of only the active group particles and the remaining ($m - d$) particles are left unchanged. This avoids the conflicting situation of the GPSO algorithm and leads the best d particles towards the optimal solution in multi-dimensional search space. We applied the OPSO algorithm to small, medium and large TGUs power system and ten selected shifted and rotated CEC 2015 benchmark functions. We have shown that the OPSO algorithm is able to achieve superior performance in terms of convergence, stability and accuracy compared to GPSO algorithm and several competitive ECTs. In the recent works, the effectiveness of the proposed OPSO algorithm has been shown for finding optimum power dispatch in smart power grid applications [53-55].

Recently, the ECTs in [18-20] have been proposed under the name, "orthogonal", i.e., orthogonal learning PSO (OLPSO) algorithms, one for local and the other for global optimization [18], orthogonal global-best-guided artificial bee colony algorithm [19], and orthogonal genetic algorithm with quantization [20]. They are using a different approach called, orthogonal experimental design (OED). The OED allows the inputs interact among them such that the output process can be optimized. The OED works on a predefined table of an orthogonal array of N factors with Q levels per factor. The OED is applied to obtain a set of possible solutions to achieve the optimal solution. However, the drawbacks through applying OED are: firstly, it holds only when no or weak interaction among the factors exists; secondly, the table that contains variable designs is complicated; thirdly, the orthogonality may not be possible achieve in the complex problems.

The rest of the paper is organized as follows. Section 2 describes the non-convex ED problem under various power constraints. Explanation of the learning strategy of the GPSO algorithm is presented in Section 3. Details of the proposed OPSO algorithm are provided in Section 4. In Section 5, the application of the OPSO algorithm to ED problem for three power systems is presented. In Section 6, the application of the OPSO algorithm to ten selected CEC 2015 benchmark functions is presented. Finally, conclusion of this study is given in Section 7.

2. Problem formulation

Here, we explain the cost function and various practical power constraints involved in this study.

2.1 Cost objective function

The main purpose of ED problem is to estimate the optimum arrangement of on-line TGUs generation in order to minimize the entire generation fuel cost subjected to the on-line TGUs and power grid constraints. The fuel cost function of each on-line TGU is characterized by quadratic function [25] and given by:

$$F(P_j) = a_j + b_j P_j + c_j P_j^2 \tag{1}$$

where $F(P_j)$ is the fuel cost function of jth TGU in (\$/h), P_j is the output active power of jth TGU in (MW), a_j, b_j, and c_j are fuel cost coefficients of jth TGU. The total fuel cost of the on-line TGUs is given by

$$\text{Minimize} \quad F_{cost} = \sum_{j=1}^{N_{gen}} F(P_j) \tag{2}$$

where N_{gen} is the number of committed on-line TGUs. F_{cost} is the function to be minimized.

2.2 Power constraints

Different practical power constraints imposed on on-line TGUs and by SPG used in the literature are explained below.

2.2.1 Power balance constraint: The total output power of committed on-line TGUs should be able to satisfy load demand and transmission network loss. The power balance constraint is given as

$$\sum_{j=1}^{N_{gen}} P_j = P_D + P_L \tag{3}$$

where P_D is load demand in (MW) and P_L is transmission network loss in (MW).

2.2.2 Transmission network loss: The transmission network loss P_L is a critical constraint of the ED problem. Not only is it desired that the power loss incurred in the system be minimized along with the total fuel cost, but the system must also generate enough power to satisfy the load demand as well as to compensate for the P_L. The P_L is given by [38,56]:

$$P_L = \sum_{j=1}^{N_{gen}} \sum_{k=1}^{N_{gen}} P_j B_{jk} P_k \tag{4}$$

where B_{jk}, are known as the loss coefficients or B-coefficients [56-57].

2.2.3 Transmission network loss mismatch: The total power generated is obtained using (3). From (3), the P_L is obtained and let's call P_{L1} as follows

$$P_{L1} = \sum_{j=1}^{N_{gen}} P_j - P_D \tag{5}$$

Besides, the P_L is also computed using (4) and let's denote this time by P_{L2} as follows:

$$P_{L2} = \sum_{j=1}^{N_{gen}} \sum_{k=1}^{N_{gen}} P_j B_{jk} P_k \tag{6}$$

Then, the difference between two, P_{L2} and P_{L1}, is called transmission network loss mismatch ($P_{L,mismatch}$) as follows:

$$P_{L,mismatch} = P_{L2} - P_{L1} \tag{7}$$

The significance of the $P_{L,mismatch}$ can be expressed as follows:

1. The effectiveness and the accuracy of the algorithm in computing optimal P_j can be determined.
2. One can determine whether or not the power balance constraint (3) is satisfied.

When $P_{L,mismatch} = 0$, then $P_{L2} = P_{L1}$. In such case, the (5) can be written as

$$P_{L2} = \sum_{j=1}^{N_{gen}} P_j - P_D \tag{8}$$

Subsequently, the (3) is satisfied.

2.2.4 Generation limits: Each TGU has a specified range within which its operation is stable. Therefore, it is desired that the TGUs be run within this range in order to maintain system stability. The generation limits of the *j*th TGU is given by

$$P_{j,min} \le P_j \le P_{j,max} \qquad j = 1, 2, ..., N_{gen} \tag{9}$$

In other words, the power generation of each TGU must remain between its minimum $P_{j,min}$ and its maximum $P_{j,max}$ limits.

2.2.5 Ramp rate limits: The operating range of all on-line TGUs is restricted by their ramp rate limits RRLs due to the physical limitation of TGUs [58-59]. For any sudden change in the load demand, TGUs increase or decrease their generation in order to satisfy the power balance constraint (3). However, the TGUs can change their generation only at a certain rate determined by the up-ramping and down-ramping rate. If a TGU is operating at a specific point, then its point of operation can be changed only up to a certain rate determined by the ramp rate. Therefore, a change in TGU output power from one specific interval to the next cannot exceed a specified limit.

If power generation need to increase, then per unit time rate of increase $P_j - P_j^0$ must satisfy

$$P_j - P_j^0 \le UR_j \tag{10}$$

If power generation need to decrease, then per unit time rate of decrease $P_j^0 - P_j$ must satisfy

$$P_j^0 - P_j \le DR_j \tag{11}$$

where P_j^0 is the TGU output active power at the previous time interval and P_j is the TGU output power at current time interval. The UR_j and DR_j are the up-ramp and down-ramp limits of TGU *j*, respectively, in (MW/h). By substituting (10) and (11) in (9), the (12) is obtained.

$$max\{P_{j,min}, (P_j^0 - DR_j)\} \le P_j \le min\{P_{j,max}, (P_j^0 + UR_j)\} \tag{12}$$

Let us assume that,

$$P_{j,low} = max\{P_{j,min}, (P_j^0 - DR_j)\}, \text{ and} \tag{13}$$

$$P_{j,high} = min\{P_{j,max}, (P_j^0 + UR_j)\} \tag{14}$$

where $P_{j,low}$ and $P_{j,high}$ are the new lower and higher limits of unit *j*, respectively.

2.2.6 Prohibited operating zones: The physical limitations due to the steam valve operation or vibration in a shaft bearing of TGU may result in the generation units operating within prohibited operating zones (POZs) [60]. The POZs make the cost function discontinuous in nature. In such case, it is difficult to determine the shape of the cost curve under POZs through actual performance testing. In addition, if the TGU operates within the POZs range in the SPG then it may lead to loss of stability. Therefore, these regions are usually avoided during generation. By using (9) mentioned in constraint 2.2.4, the feasible operating zones FOZs of the *j*th TGU are given by

$$
\begin{aligned}
&P_{j,min} \le P_j \le P_{j,1}^{l} \\
&P_{j,k-1}^{u} \le P_j \le P_{j,k}^{l} \qquad k = 2, 3, ..., N_{j,PZ} \\
&P_{j,N_{j,PZ}}^{u} \le P_j \le P_{j,max}
\end{aligned}
\tag{15}
$$

where $P_{j,k}^{l}$ and $P_{j,k}^{u}$ are the lower and upper bound of the *k*th POZs of the *j*th unit, and $N_{j,PZ}$ is number of POZs of the *j*th TGU. Incorporating these power constraints in (12) - (15), we get the final set of inequality power constraints imposed on TGU in SPG are as follows:

$$
\begin{aligned}
&P_{j,low} \le P_j \le P_{j,1}^{l}, \\
&P_{j,k-1}^{u} \le P_j \le P_{j,k}^{l} \qquad k = 2, 3, ..., N_{j,PZ} \\
&P_{j,N_{j,PZ}}^{u} \le P_j \le P_{j,high}
\end{aligned}
\tag{16}
$$

Observation:

Equation (16) gives the final set of the inequality power constraints imposed on *j*th TGU in terms of new lower and upper generation limits with RRLs and FOZs. In addition, the (16) avoids all POZ_S imposed on *j*th TGU. Thus, all TGUs will have operation limits (OLs) satisfying all power constraints.

An Illustrative Example: In order to illustrate new lower and upper generation limits and FOZs generated due to presence RRLs and POZs of *j*th TGU, an example of specifications of TGU_2 per one hour generation is given below [25]:

P_2^0 = 170 MW; $P_{2,min}$ = 50 MW; $P_{2,max}$ = 200 MW; UR_2 = 50 MW; DR_2 = 90 MW. The TGU_2 has two POZs are: POZ_1 = [90,110] and POZ_2 = [140,160].

From (16), the new lower and upper limits of TGU_2 based on RRLs are:

$P_{2,low}$ = 80 MW and $P_{2,high}$ = 200 MW,

and there are three FOZs are:

FOZ_1: $80 \le P_2 \le 90$

FOZ_2: $110 \le P_2 \le 140$

FOZ_3: $160 \le P_2 \le 200$

Fig. 1 shows that TGU_2 has minimum and maximum OLs given by 50 MW and 200 MW, respectively. However, due to presence RRLs (up-ramp and down-ramp limits) power constraint, TGU_2 operates within new lower and higher OLs given by $P_{2,low}$ = 80 MM and $P_{2,high}$ = 200 MW. In addition, three FOZs are: FOZ_1 = [80,90] MW, FOZ_2 = [110,160] MW, FOZ_3 = [160,200] MW in white color, and two POZs are: POZ_1 = [90,110] MW, POZ_2 = [140,160] MW in dark color are shown in Fig. 1. The intermittent zone ([50,80] MW) is out of OL of the TGU_2.

3. The GPSO algorithm

The mechanism of GPSO algorithm depends on distribution of the particles (possible solutions) in a swarm. Firstly, each particle flying in the multi-dimensional search area adjusts its flying trajectory according to two guides, its personal experience ($G_{pers,i}$) and its neighborhood's best experience (G_{best}). Secondly, when seeking a global solution, each particle learns from its own historical experience and its neighborhood's historical experience. In such a case, a particle while choosing the neighborhood's best experience uses the best experience of the whole swarm as its neighbor's best experience. Since the position of each particle is affected by the best-fit particle in the entire swarm, this technique is named, global PSO [16-17], [61-64]. Only a few parameters have been used in GPSO algorithm later to give potential advantage and to enhance its performance. Among user parameters of GPSO algorithm, several strategies of inertia weight have been used. For example, constant inertia weight [61], time-varying inertia weight [62] and constriction factor for balancing global and local searches [63]. However, when the cost function is within high-dimensional search space and restricted by many POZs, this makes the cost function has multiple local minima. Then, finding global optimum becomes more difficult with these few parameters, because these few parameters become an ineffective. Therefore, the original GPSO algorithm is considered as a fundamental technique of PSO algorithm. The following steps explain the mechanism of the GPSO algorithm.

Let us consider a swarm population with m particles searching for a solution in d-dimensional search space, where $m > 1$. The objective of the GPSO algorithm is to minimize the given objective function $f(x)$. Each particle i ($i = 1, 2, ..., m$) is associated with two d-dimensional vectors; a velocity vector V_i and a position vector X_i.

***Initialization*:** Iteration, $t = 0$.

***Step* 1:** Initialize the velocity and position vectors randomly for particle i, ($i = 1, 2, ..., m$).

$$V_i(0) = [v_{i1}, v_{i2}, \ldots, v_{id}] \tag{17}$$

$$X_i(0) = [x_{i1}, x_{i2}, \ldots, x_{id}] \tag{18}$$

***Step* 2:** For each particle i, evaluate the objective function $f(x)$ using the position vector $X_i(0)$.

***Step* 3:** Initialize the personal position vector of particle i, $G_{pers,i}(0)$ as follows:

$$G_{pers,i}(0) = [g_{pi,1}, g_{pi,2}, \ldots, g_{pi,d}] = X_i(0) \tag{19}$$

***Step* 4:** Determine the global best position vector, $G_{best}(0)$. It is the best position vector among all personal positions vectors in the swarm. The $G_{best}(0)$ is given by

$$G_{best}(0) = [g_{b,1}, g_{b,2}, \ldots, g_{b,d}] \tag{20}$$

***Update*:** Iteration, t = 1, 2, ..., N_{iter}, where N_{iter} is the total number of iterations.

***Step* 5:** In iteration t, the particle's velocity and position vectors are updated as follows:

$$V_i(t) = V_i(t-1) + c_1 r_1(t) \left(G_{pers,i}(t-1) - X_i(t-1)\right) + c_2 r_2(t) \left(G_{best}(t-1) - X_i(t-1)\right) \tag{21}$$

$$X_i(t) = X_i(t-1) + V_i(t) \tag{22}$$

where c_1 and c_2 are two positive acceleration coefficients whose values are chosen by using trail and error. The range of c_1 and c_2 are of [2.00,2.25]. They are adapting controlled based on the evolutionary states [64-66]. However, in most cases, $c_1 = c_2 = 2.00$. The $r_1(t)$ and $r_2(t)$ are two randomly generated values within the range (0,1).

***Step* 6:** Each particle i, the objective function $f(x)$ is evaluated using the position vector $X_i(t)$.

***Step* 7:** The $G_{pers,i}$ and G_{best} are updated as follows:

$$G_{pers,i}(t) = \begin{cases} X_i(t) & if \;\; f(X_i(t)) \le f(G_{pers,i}(t-1)) \\ G_{pers,i}(t-1) & \text{Otherwise} \end{cases} \tag{23}$$

$$G_{best}(t) = min\{G_{pers,i}(t)\} \tag{24}$$

End of iteration, $t = N_{iter}$

***Step* 8:** The global best position vector $G_{best}(t)$ becomes the optimal solution and the $f(G_{best}(t))$ gives the optimal value of the objective function. A flowchart of the GPSO algorithm is shown in Fig. 2.

4. The OPSO algorithm

Here, the details of the proposed OPSO algorithm and explanation of the diagonalization process are provided.

4.1 Orthogonal diagonalization process

The matrix diagonalization is the process of converting a square matrix, B of size ($d\times d$), into a diagonal matrix, D of size ($d\times d$), as shown below [67].

$$B = Q\, D\, Q^{-1} \tag{25}$$

where Q is a matrix of size ($d\times d$) composed of eigenvectors of B and the diagonal elements of D contains the corresponding eigenvalues. The Q is an invertible matrix because it contains linearly independent vectors. When B is symmetric, the (25) may be written as

$$B = C\, D\, C^{-1} \tag{26}$$

in which the columns of matrix C are orthonormal to each other. The (26) can be rewritten as

$$D = C^{-1}\, B\, C \tag{27}$$

Since matrix C is an orthonormal matrix, the (27) can be written as

$$D = C^{\mathrm{T}} B\, C \tag{28}$$

Equation (28) is called orthogonal diagonalization (OD). The process of OD is shown in Fig. 3.

4.2 OPSO learning algorithm

In this paper, an orthogonal PSO (OPSO) algorithm is proposed to improve the performance of the GPSO algorithm. The objective of the OPSO algorithm is to minimize the given d-dimensional objective function $f(x)$. Consider a swarm population with m particles, each with a dimension of d ($d \le m$). The OPSO algorithm provides a new topology to the swarm population. In each iteration, the swarm population of m particles are divided into two groups: an active group that consists of best personal experience of d particles and one passive group which consists of the personal experience of rest ($m - d$) particles. The opinion of the active group particles are honoured by updating their respective velocity and position vectors. The opinion of the passive group particles are ignored because their guidance may be erratic, and therefore, their velocity and position vectors are not updated. In each iteration, the OD process (28) is applied. The matrix B is obtained from the d best particles of the active group and thereafter, orthonormal matrix C and diagonal matrix D are computed using (28). The steps involved in OPSO algorithm are given below.

Let a d-dimensional objective function $f(x)$ need to be optimized.

***Initialization*:** Iteration, $t = 0$.

***Step* 1:** Randomly initialize the velocity $V_i(0)$ and position $X_i(0)$ vectors for particle i, ($i = 1, 2, ..., m$).

***Step* 2:** Determine the personal position vectors, $G_{pers,i}(0)$ using (19).

***Step* 3:** Evaluate the objective function $f(x)$ using position vector $X_i(0)$.

OD process:

In iteration t, $t = 1, 2, ..., N_{iter}$, where N_{iter} is the total number of iterations.

***Step* 4:** Arrange the m personal position vectors in an ascending order based on their $f(x)$ values.

***Step* 5:** Construct matrix A of size ($m\times d$) such that each row occupies one of the m personal position vectors in the same ordered sequence as in step 4.

***Step* 6:** Using pseudocode given in Fig. 4, convert matrix A to a symmetric matrix B of size ($d\times d$), such that B is a real symmetric matrix of dimension ($d\times d$).

***Step* 7:** Apply the OD process shown in Fig. 3 on matrix B to obtain the diagonal matrix D. Let D_i denote the ith row of matrix D, where $i = 1, 2, ..., d$.

***Update*:**

***Step* 8:** Update the position and velocity vectors of the active group particles, $i = 1, 2, .., d$, as follows.

$$V_i(t) = V_i(t-1) + c\, r(t)\, [D_i(t) - X_i(t-1)] \tag{29}$$

$$X_i(t) = X_i(t-1) + V_i(t) \tag{30}$$

where c is an acceleration coefficient and is chosen by trial and error in the range [2.00,2.25] and $r(t)$ is a random number within range (0,1).

***Step* 9:** Determine the $G_{pers,i}(t)$, i = 1, 2, ..., m, as follows.

$$G_{pers,i}(t) = \begin{cases} X_i(t) & if \;\; f(X_i(t)) \le f(G_{pers,i}(t-1)) \\ G_{pers,i}(t-1) & \text{Otherwise} \end{cases} \tag{31}$$

***Step* 10:** Determine the global best position, $G_{best}(t)$.

$$G_{best}(t) = min\{G_{pers,i}(t)\},\; i = 1, 2, ..., m \tag{32}$$

End of iteration, $t = N_{iter}$

***Step* 11:** The $G_{best}(N_{iter})$ as computed in step 10 provides the optimal solution. A flowchart of the OPSO algorithm is shown in Fig. 5.

***Observation* 1:** One of the important observation of the OPSO algorithm is as follows. Sine D is a diagonal matrix, the d rows or columns of matrix D are orthogonal vectors. These d vectors are used to diminish the contribution of $X_i(t–1)$ while updating $V_i(t)$, i = 1, 2, ..., d. As $t\rightarrow\infty$, assume that the algorithm has converged. In such case, (30) can be written as:

$$\lim_{t\to\infty} X_i(t) = X_i(t-1) \tag{33}$$

This implies that $\lim_{t\to\infty} V_i(t) = 0$. Then, (29) can be written as:

$$\lim_{t\to\infty} V_i(t) = V_i(t-1) = 0 \tag{34}$$

which implies that

$$\lim_{t\to\infty} c\, r(t)\, [D_i(t) - X_i(t-1)] = 0 \tag{35}$$

Since $c\, r(t)$ is constant,

$$\lim_{t\to\infty} X_i(t-1) = D_i(t) \tag{36}$$

From (36) it is evident that $\lim_{t\to\infty} X_i(t)$ becomes diagonal and equals to D_i when iteration becomes large and the algorithm has converged.

Considering the d positions vectors, (36) can be written in matrix form as:

$$[X(t)]_{\text{active\_group}} = [D(t)]_{\text{active\_group}} \tag{37}$$

This means, at $t\rightarrow\infty$, the d position vectors of the active group particles are equal to the d orthogonal vectors in matrix D. Thus, the OPSO algorithm reaches convergence and leads to optimal solution.

***Observation* 2:** In case of GPSO algorithm (21), two guides, $G_{pers,i}$ and G_{best}, are used to update the velocity vector $V_i(t)$. These two guides may conflict each other which leads to zigzag behaviour of the algorithm, that in turn causes trapping into a local minima. Whereas, in case of OPSO algorithm (29), as only one guide, $D_i(t)$ is used in updating of the velocity vector, $V_i(t)$, such situation is eliminated.

***Observation* 3:** Due to the orthogonalization of the position vectors, as iteration progresses, once the optimal solution is achieved, the solution remains the same in the subsequent iterations until the end of the total number of iterations. This fact provides stability to the proposed algorithm.

***Observation* 4**: From (29), the velocity vector of each particle of active group can be rewritten as follows.

$$\begin{aligned} V_1(t) &= V_1(t-1) + cr(t)[D_1(t) - X_1(t-1)] \\ V_2(t) &= V_2(t-1) + cr(t)[D_2(t) - X_2(t-1)] \\ \vdots & \quad \vdots \qquad\qquad \vdots \qquad\qquad \vdots \\ V_d(t) &= V_d(t-1) + cr(t)[D_d(t) - X_d(t-1)] \end{aligned} \tag{38}$$

where

$$\begin{aligned} D_1(t) &= [d_{11}, \quad 0, \quad 0, \cdots, 0] \\ D_2(t) &= [0, \quad d_{22}, \quad 0, \cdots, 0] \\ \vdots \quad & \quad \vdots \qquad \vdots \qquad \vdots \qquad \vdots \\ D_d(t) &= [0, \quad 0, \quad 0, \quad d_{dd}] \end{aligned} \tag{39}$$

It can be seen by cooperation (38) and (39), the position vector X_i, i = 1, 2, ..., d, of active group is affected by only one orthogonal vector D_i, i = 1, 2, ..., d. Thus, while updating, each V_i, i =1, 2, ..., d, is perturbed only by d_{ii} in the d-dimensional search space. Due to this, the OPSO algorithm gives faster convergence and better solution.

***Observation* 5**: When $m = d$, there is no existence of passive group and therefore we do not see any advantages of diversity and the solution may not yield the best. When $m >> d$, it gives rise to more computation, but does not provides any better solution, as seen from sensitivity analysis (Section 6.6). Considering these two extremes a reasonable value of m is could be about 10% more than d.

***An Illustrative Example*:** In order to explain the mechanism of OPSO algorithm, Figure 6 illustrates an example of a 2-dimensional shifted function, $f(x,y) = (x - 2)^2 + (y + 3)^2 + 9$. From visual inspection, it can be seen that the x and y are shifted from the origin [0,0] by [2.0,–3.0]. The optimum solution of the given function equals to 9.0 at [x,y] = [2.0,–3.0]. The aim of the algorithm is to find the values x and y such that the $f(x,y)$ is minimized.

The OPSO algorithm program is implemented using MATLAB software in a personal computer with the following specifications: Intel (R) core (TM) 2 Duo CPU T6570 @ 2.1 GHz. RAM of 4GB and Windows 7, 64-bit operating system. The OPSO algorithm is executed with m = 6 for N_{iter} = 200. The values of position vectors (X_i, i = 1, 2, ..., 6), the diagonal vectors (D_i, i = 1, 2) and personal vectors ($G_{pers,i}$, i = 1, 2, ..., 6) at iteration t = 1, 80, 140 and 200 are shown in Fig. 7. The six particles are divided into an active group of two best particles and a passive group of remaining four particles. According to the OD process, $G_{pers,1}$ and $G_{pers,2}$ are assigned to the active group and ($G_{pers,3}$, ..., $G_{pers,6}$) are assigned to the passive group. In each iteration, the velocity and position vectors of only the active group are updated. As seen from Figure 7, as iteration reaches 200, the OD process causes $[X]_{\text{active\_group}} = [D]_{\text{active\_group}}$, satisfying (37) and causing X to be a diagonal matrix. At the end of iteration, the best G_{pers}, provides the optimal solution i.e., G_{best} = [2.0,–3.0].

Figure 8 geometrically explain the movement of the six particles of the swarm as iteration progress from 1 to 200. At first iteration, the X_i vectors are at random positions in search space range of ± 100. The vectors D_1 and D_2 are orthogonal to each other. As the iteration increases, the active group position vectors (X_1 and X_2) moves towards the orthogonal vectors (D_1 and D_2) and at the end of iteration, $X_1 = D_1$ and $X_2 = D_2$. This can be seen from bottom subfigure of Figure 8 which is the magnified view at t = 200. This signifies achievements of orthogonalization of the active group position vectors. Figure 9 shows the movement of the personal position vectors of six particles $G_{pers,i}$, i = 1, 2, ..., 6 and global best position vector G_{best}, two active group particles (1 and 2) and four passive group particles (3, 4, 5 and 6) searching for global solution [2.0,–3.0]. The best personal vector, i.e., G_{best} of active group gives the optimal solution as shown in Figure 9.

5. Application of OPSO algorithm to ED problem

Here we describe the simulation results carried out on three power systems with several TGUs and SPG constraints.

5.1 Performance Measures

To study the accuracy, stability and robustness of different algorithms, several fitness values as explained below are considered. Every algorithm is executed over N_{run} runs each with N_{iter} iterations.

1. Minimum fuel cost (F_{min}): Defined as the minimum of the optimized F_{cost} values obtained from N_{run} independent runs.

2. Maximum fuel cost (F_{max}): Defined as the maximum of the optimized F_{cost} values obtained from N_{run} independent runs.

3. Mean fuel cost (F_{mean}): Defined as the average of the optimized F_{cost} values obtained from N_{run} independent runs.

4. Standard deviation (σ): The σ is the standard deviation of the optimized F_{cost} values obtained from N_{run} independent runs.
5. Range (R): The range (R) is defined as the difference between F_{max} and F_{min}.
6. Average execution time (AET): It is the time consumed by an algorithm after convergence, averaged over N_{run} independent runs.

5.2 Test Case 1: Power system-1 (PS-1)

The PS-1, as shown in Fig. 10, is a small-scale power system with six TGUs (N_{gen} = 6) and 26 buses [25]. At steady state normal operation, the maximum load demand is given as P_D = 1,263 MW. The specifications of PS-1, i.e., initial output power, generation limits, cost coefficients, RRLs and 12 POZs are given in Table 1 and the B-loss coefficients are given in Table 2.

5.2.1 Comparison in terms of fitness values

In [25], the authors have shown the superior performance of their proposed RDPSO algorithm over other 10 ECTs for the ED problem using PS-1. In addition, PS-1 was also used by other researchers for performance comparison, e.g., the SOH-PSO [3], SA-PSO [26], NPSO-LRS [27], CPSO [29], QMPSO [30], FDA [38], GA-API [40], MIQCQP [41], λ-logic [43], SPPO [46]. We compared the performance of our proposed OPSO algorithm with these 21 ECTs and GPSO algorithm. The comparison results of fitness values are shown in Table 3. The parameters used OPSO and GPSO algorithms are: m = 10, N_{run} = 100, N_{iter} = 1000, $c_1 = c_2 = c$ =2.05. Here, "NA" stands for not available. One can observe the following from this Table. Firstly, the OPSO algorithm provides best result in terms of lowest F_{mean} and lowest σ. This shows that the OPSO algorithm provides stable and accurate solution. Secondly, in terms of range, R, OPSO provides the second best result, i.e., R = \$0.3276/h (the best is from GA-API algorithm [40], R = \$0.0300/h). This indicates that OPSO algorithm has second lowest dispersion of optimum F_{cost}. However, the F_{min} and F_{max} of OPSO algorithm are much better than F_{min} and F_{max} for GA-API algorithm [40]. In term of AET, the MIQCQP [41] is best one. The GPSO algorithm is second best but it provides worst results in terms of other performance measures. The OPSO achieves the third best in terms of AET. Thus, the overall performance of the OPSO algorithm is far superior than the other 22 ECTs.

5.2.2 Convergence characteristics of OPSO and GPSO algorithms

Figure 11 shows the convergence characteristics of OPSO and GPSO algorithms for PS-1. It shows ensemble average F_{cost} values obtained from 100 independent runs at each iteration. It can be seen that OPSO algorithm settles approximately at 40 iterations and achieved F_{mean} = \$15,443.5921/h whereas GPSO algorithm takes about 190 iterations to converge and achieved F_{mean} = \$15,460.8461/h. This shows faster convergence of OPSO algorithm compared with the GPSO algorithm.

Figure 12 shows the comparison of optimized F_{cost} at each run between the OPSO and GPSO algorithms. In case of OPSO algorithm, the optimized F_{cost} after each run remains more or less steady at about \$15,443/h, whereas in GPSO algorithm, the optimized F_{cost} varies between \$15,442.8326/h and \$16,103.3400/h. This indicates that OPSO algorithm is more consistent and stable than GPSO algorithm.

5.2.3 Comparison in terms of inequality constraint

Table 4 lists the solution vector, P_j (j = 1, 2, ..., 6) corresponding to the best solution for OPSO and GPSO algorithms. The results in Table 4 observe that OPSO and GPSO algorithms avoid the 12 POZs of 6 TGUs and are within RRLs (16). This indicates that both algorithms are able to satisfy the inequality constraints of PS-1.

5.2.4 Comparison in terms of power balance constraint

Table 5 shows the results of power balance constraint based on $P_{L,mismatch}$ for 11 ECTs. The load demand of PS-1 is given as P_D = 1,263 MW. Using the total optimum output power generated (from Table 4) and equations (5)-(7), P_{L1}, P_{L2} and $P_{L,mismatch}$ are computed and the results presented in Table 5. It can be seen that OPSO as well as SOH-PSO [3], GA-API [40], MIQCQP [41], λ-logic [43] and SPPO [46] algorithms provide zero mismatch, i.e., $P_{L,mismatch} = 0$, indicating that the power balance constraint, that is equation (3) is satisfied.

5.3 Test Case 2: Power system-2 (PS-2)

The PS-2 is a medium-scale power system [29] with 15 TGUs (N_{gen} =15) whose generation specifications and B-loss coefficients are shown in Table 6 and Table 7, respectively. The maximum load demand at steady state normal operation is given as P_D = 2,630 MW. The PS-2 has 11 POZs in 4 TGUs (2, 5, 6 and 12) and RRLs for each TGU.

5.3.1 Comparison in terms of fitness values

In [25], the RDPSO algorithm was tested with PS-2 and its superior performance compared to other 10 ECTs has been shown. In addition, the PSO-MSAF [22], SA-PSO [26], CPSO [29], EGSSOA [42], λ-logic [43], SPPO [46] algorithms have also been tested with PS-2. Here we compare performance of OPSO algorithm with GPSO and other existing 17 ECTs. The set of parameters used in GPSO and OPSO algorithms are: m = 18, d = 15, N_{run} = 100, N_{iter} = 1000 and $c = c_1 = c_2 = 2.05$. Comparison of fitness values between OPSO algorithm and other 18 existing ECTs are listed in Table 8. It can be seen that, the OPSO algorithm provides the best results in terms of F_{mean}, σ and R. These results indicate that the OPSO algorithm provides consistent, stable and optimal results. However, in term of AET, OPSO is the second best; the GPSO algorithm being the best among the 19 ECTs.

5.3.2 Convergence characteristics of OPSO and GPSO algorithms

Figure 13 shows the convergence characteristics of OPSO and GPSO algorithms for PS-2. It shows ensemble average F_{cost} values at each iteration obtained from 100 independent runs. It can be seen that OPSO algorithm settles at about 60 iterations to achieve F_{mean} = \$32,670/h whereas GPSO algorithm takes about 150 iterations to settle and achieved F_{mean} = \$33,185/h.

Figure 14 shows the distribution of optimized F_{cost} at each run. It shows that the optimized F_{cost} of OPSO remains steady at about \$32,669/h, whereas in GPSO algorithm, the optimized F_{cost} varies over a wide range from \$32,892/h and \$33,851/h. This indicates that OPSO algorithm is more consistent, stable and reliable than the GPSO algorithm.

5.3.3 Comparison in terms of inequality constraint

Table 9 presents the solution vector, P_j (j = 1, 2, ..., 15) corresponding to the best solution for OPSO and GPSO algorithms. It can be seen that both the OPSO and GPSO algorithms are able to avoid the 11 POZs of 4 TGUs and are within RRL constraints, thus both algorithms are able to satisfy the inequality constraints of PS-2.

5.3.4 Comparison in terms of power balance constraint

Table 10 shows results of power balance constraints for the OPSO and other 8 ECTs. The load demand of PS-2 is given as P_D = 2,630 MW. Using the optimum output power generated given in Table 9 and Equations (5)-(7), P_{L1}, P_{L2} and $P_{L,mismatch}$, were determined. It can be seen that the OPSO algorithm as well as PSO-MSAF [22], EGSSOA [42] and λ-logic [43] algorithm are satisfying the zero mismatch condition, i.e., $P_{L,mismatch}$ = 0, thus satisfying (3).

5.4 Test Case 3: Power system-3 (PS-3)

The PS-3 is a large-scale power system taken from Taipower system [9], [25]. It consists of 40 mixed-generating units, coal-fired, gas-fired, gas-turbines with complex cycle, diesel generating units and nuclear generating units. The maximum load demand at steady state normal operation is given as P_D = 8,550 MW. The PS-3 contains 46 POZs distributed among 25 TGUs and are shown in Table 11. The RRLs are imposed on all the 40 TGUs. The *B*-loss coefficients are considered and they are generated randomly as is done in [43]. The *B*-loss coefficients matrix of dimension 40×40 is listed in Appendix. Unfortunately, the PS-3 is tested by only a few authors under RRLs, POZs and P_L constraint. This may be due to unavailability of *B*-loss coefficients or due to its high dimension with a large number of power constraints.

5.4.1 Comparison in terms of fitness values

In [25], PS-3 has been tested with 11 ECTs and superior performance of RDPSO algorithm over other 10 ECTs has been shown. However, the 46 POZs, RRLs of each TGU and the P_L constraint have not been considered. Therefore, these results are less constrained. Considering all the POZs, RRLs and P_L constraint, the PS-3 has been tested by MIQCQP [41], λ-logic [43], the proposed OPSO and GPSO algorithms. Thus, the performance of OPSO algorithm and other existing 14 ECTs are compared. The set parameters used in GPSO and OPSO algorithms are: m = 45, N_{run} = 100, N_{iter} = 10,000, d = 40, and $c = c_1 = c_2 = 2.05$. The fitness values of the 15 ECTs are listed in Table 12. It can be seen that the OPSO algorithm provides the best result in terms of F_{mean} and σ over 100 independent runs. This indicated that the OPSO algorithm provides the most optimal and consistent results. In addition, the range R of OPSO algorithm is the lowest among the 15 ECTs, thus indicating that OPSO algorithm provides solution with the lowest dispersion. Since the AET values are not available for other ECTs, between OPSO and GPSO algorithms, the AET of OPSO (69 sec), due to its computational complexity, is found to be higher than the GPSO (47 sec). These results indicate that among the 15 ECTs, the OPSO algorithm is the most stable, robust and is able to provide most optimal solution.

5.4.2 Convergence characteristics of OPSO and GPSO algorithms

Figure 15 shows the convergence characteristics of OPSO and GPSO algorithms for PS-3. Note that Figure 14 is drawn with N_{iter} = 2000, to give a better visualization. However, the OPSO and GPSO algorithms run with N_{iter} = 10,000. It shows ensemble average F_{cost} values at each iteration obtained from 100 independent runs. It can be seen that OPSO algorithm settles at about 1,600 iterations and achieves F_{mean} of about \$127,997/h. Whereas, the GPSO algorithm takes about 1000 iterations to converge, but settles at a local minimum with a non-optimal F_{mean} of about \$398,709/h. This indicates that the GPSO algorithm is unable to solve ED problem with such a high dimension and under such large number of power constraints. In contrast, the OPSO algorithm gives high accuracy in solving such this complex problem.

Figure 16 shows the variation of optimized F_{cost} over 100 independent runs achieved by the OPSO and GPSO algorithms. It shows that the optimized F_{cost} of OPSO varies between \$126,489.6/h and \$127,916.2/h, whereas in GPSO algorithm, it varies between \$139,051.2/h and \$515,712.5/h. This indicates that OPSO algorithm is capable of providing consistent and reliable optimal solution. Whereas, the GPSO algorithm is unable to provide optimal solution due to the high complexity of the problem.

5.4.3 Comparison in terms of inequality constraint

Table 13 presents solution vector, P_j (j = 1, 2, ..., 40) corresponding to the best solution obtained from the OPSO and GPSO algorithms. In case of GPSO algorithm, the TGU_4 violates RRLs (red color). The TGU_4 must operate within $P_{4,low}$ = 24 MW and $P_{4,high}$ = 42 MW (16). This means that GPSO algorithm fails in solving PS-3 indicating that GPSO algorithm is unable to solve large scale ED problem. Whereas, the OPSO algorithm avoids the 46 POZs of 25 TGUs and is within RRLs.

5.4.4 Comparison in terms of power balance constraint

Since all the data for other existing ECTs are not available for PS-3, we compare the performance between OPSO algorithm and λ-logic [43]. The GPSO is out of comparison, because it failed in solving PS-3. The load demand of PS-3 is given as P_D = 8,550 MW. Using the total optimum output power generated (Table 13) and Equations (5)-(7), P_{L1}, P_{L2} and $P_{L,mismatch}$, are determined and are presented in Table 14. It can be seen that $P_{L,mismatch}$ of OPSO algorithm is more close to 0.0 than λ-logic [43], which indicates better performance of OPSO algorithm.

6. Application of OPSO algorithm to CEC 2015 benchmark functions.

Economic dispatch of power under various power constraints makes the objective function, i.e., cost function becomes multimodal function and it has multiple local optimums as discussed in Section 1. In order to provide a fair comparison and demonstrate the goodness of the proposed OPSO algorithm, ten selected shifted and rotated functions from CEC benchmark functions 2015 are added. Here we describe these functions and investigate performance of the GPSO and OPSO algorithms along with a few competitive ECTs.

6.1 Benchmark functions

Ten benchmark functions listed in Table 15 are used in this study. These benchmark functions are taken from the congress on evolutionary computation (CEC) 2015 and are used in performance comparison of global optimization algorithms [31-32]. All ten benchmark functions are minimization tasks. In addition, these are shifted and rotated functions, the global optimum solution x as shown in Table 15 is not located in the center of the search domain. The optimum solution x is shifted to a new position vector, i.e., shifted global optimum, $O_{ji} = [o_{j1}, o_{j2}, . . ., o_{jd}]$, where j = 1, 2, ..., 10 and i = 1, 2, ..., d. the d is the dimension of the each benchmark function. As well as, all functions are rotated by rotation matrix, M_j, j =1, 2, ..., 10. The rotation does not affect the shape of the function but increases the function complexity in finding global optimum. The M is $d \times d$ matrix. It is applied to obtain the rotation and is generated from standard normally distributed entries using Gram-Schmidt orthnormalization process [31-32].

The ten benchmark functions are divided into two groups based on their significant physical properties. The first group involves three unimodal benchmark functions $f_1 - f_3$ [31-32]. These are f_1 (Shifted and Rotated High Conditioned Elliptic), f_2 (Shifted and Rotated Cigar) and f_3 (Shifted and Rotated Discus). The second group includes seven multimodal benchmark functions $f_4 - f_{10}$. Finding global optimum solution G_{best} is more interesting since these benchmark functions are more difficult to optimize because of the number of local minima as well as they are shifted and rotated. In multimodal functions, the number of local minima increases as the problem dimension increases [6], [43]. Therefore, the search algorithm should be able to obtain a good solution and not become trapped in a local minimum. The seven multimodal functions are f_4 (Shifted and Rotated Ackley), f_5 (Shifted and Rotated Weierstrass), f_6 (Shifted and Rotated Rastrigin), f_7 (Shifted and Rotated Katsuura), f_8 (Shifted and Rotated HappyCat), f_9 (Shifted and Rotated HGBat) and f_{10} (Shifted and Rotated Expanded Griewank plus Rosenbrock).

Table 15 shows the details of the ten selected CEC 2015 benchmark functions. The name and mathematical description of f_1–f_{10} are shown in columns 2 and 3, respectively. The "Threshold Error" value of each function is available in column 4. The "optimum x" in column 5 and the minimum value of each function, "minimum $f(x)$" is in column 6. The solution of each function is judged successful, when the algorithm reaches to a value smaller than "Threshold Error". In other words, the algorithm passes the test.

6.2 Performance measures and experimental setup

To study the accuracy, stability and reliability of different algorithms, nine performance measures as explained below are considered. Let m be the number of particles in the swarm. Each algorithm is run over N_{run} times for N_{iter} iterations.

1. Number of Function Evaluations (NFE): The NFE is used as a measure of computational complexity of the algorithms. The NFE is the number of times the objective function $f(x)$ is evaluated in one run of the algorithm and is given by

$$\text{NFE} = m \times N_{iter} \tag{40}$$

2. Best Fitness Value (BFV): The BFV is defined as the minimum optimized $f(x)$ value obtained from N_{run} independent runs.

3. Worst Fitness Value (WFV): The WFV is defined as the maximum optimized $f(x)$ value obtained from N_{run} independent runs.

4. Mean Fitness Value (MFV): The MFV is defined as the average of the N_{run} BFVs.

5. Minimum Function Error Value (MFEV): The MFEV is defined as the difference between minimum f(x), i.e., column 6 in Table 15 and MFV. The MFEV is given by

$$\text{MFEV} = \left| \text{minimum} f(x) - \text{MFV} \right| \tag{41}$$

6. Standard deviation (σ): The σ is the standard deviation of the N_{run} BFVs.

7. Success Rate (SR): An algorithm is successful if the MFEV of each function falls below the "Threshold Error". The SR is used as a measure of reliability of the algorithm [31-32]. The SR in percentage is given by

$$\text{SR} = \frac{\text{Number of successful runs}}{N_{run}} \times 100 \tag{42}$$

8. Reliability Rate (RR): The RR of an algorithm over all the ten benchmark functions is defined

$$\mathrm{RR} = \frac{1}{10}\sum_{i=1}^{10}\mathrm{SR}_i \tag{43}$$

where SR_i is the success rate of the benchmark function $f_i(x)$, i = 1, 2, ..., 10.

9. Average execution time (AET): It is the time consumed by an algorithm until it reaches to MFEV, averaged over N_{run} independent runs.

In order to measure the accuracy, stability and robustness of each algorithm, the GPSO and OPSO algorithms were evaluated using the ten unimodal and multimodal functions given in Table 15. Each function is tested with 30-dimension, d =30. Based on the suggestion by the CEC 2015 [32], the optimization task has been carried out for N_{run} = 20 independent runs. The GPSO and OPSO algorithms are terminated when reaching the MFEV of each is smaller than 1.00×10^{-03}. The $N_{particle}$ in the OPSO and GPSO algorithm for f_1, f_2, ..., f_{10} is, m = 40. The OPSO algorithm requires to be $m \geq d$. However this condition is not imposed in GPSO algorithm. The number of iterations N_{iter} is obtained from (40) once maximum NFE and m is decided. Thus, N_{iter} = 3,750. Both GPSO and OPSO algorithms are run with maximum NFE = 150,000. The acceleration coefficients values of c_1 and c_2 in GPSO and c in OPSO algorithm are set at 2.00 and 2.05, respectively, using trial and error method. The parameters $r(t)$, $r_1(t)$and $r_2(t)$ are chosen randomly. In addition, the shifted global optimum vector O_{ji} for each function is randomly distributed in $[-80,80]^{30}$. In addition, an orthogonal (rotation) matrix M_j of each function is generated using Gram-Schmidt orthnormalization process.

6.3 Comparison in terms of fitness values

Performance comparison between GPSO and OPSO algorithms in terms of BFV, WFV, MFV, MFEV, σ and AET are shown in Table 16. It can be seen that in case of GPSO algorithm, the three fitness values BFV, WFV and MFV differ substantially from their optimal values for the ten functions. Whereas, in OPSO algorithm, the three fitness values are the same to their optimum values for all the ten functions. The MFEV of GPSO algorithm is so far from "Threshold Error" in f_1–f_{10}. However, in OPSO algorithm, the MFEV is smaller than "Threshold Error". It is achieved MFEV = 0.0 for the f_1–f_{10}. In terms of the standard deviation σ, it remains close to 0.0 in OPSO algorithm, indicating high stability and reliability of the OPSO algorithm. The results shown in Table 16, thus proves that OPSO algorithm is more accurate, stable and robust compared to the GPSO algorithm. In terms of the AET, the OPSO algorithm reaches "Threshold Error" within a specific AET as shown in Table 16. However, GPSO algorithm can't reach "Threshold Error", Which indicating that GPSO is unable to solve these ten shifted and rotated CEC 2015 benchmark functions under 30-dimension.

6.4 Success rate and reliability rate

The performance comparison between the GPSO and the OPSO algorithms with N_{run} = 20 (independent runs) in terms of SR and RR. The GPSO algorithm fails in ten CEC 2015 benchmark functions. However, the OPSO algorithm was successful in all these functions giving rise to SR of 100%. The RR of GPSO and OPSO algorithms are thus found to be 0.0% and 100%, respectively.

6.5 Convergence characteristics

Figure 17 shows the convergence characteristics of GPSO and OPSO algorithms for ten shifted and rotated CEC 2015 benchmark functions $f_1 - f_{10}$. The comparison is obtained in terms of fitness value (FV) averaged over N_{run} times at each NFE. It can be seen that, in case of OPSO algorithm the FV reduces to minimum value of $f(x)$) as NFE less than 4.0×10^{04} except f_1. Whereas, in case of GPSO algorithm, the FV fails to converge and remains above the "Threshold Error", which is indicating failure of the algorithm.

In order to highlight the superior performance of the OPSO algorithm over the GPSO algorithm, in Figure 18 we provide the function error value (FEV) averaged over N_{run} times at each NFE for the four selected CEC 2015 benchmark functions, f_1, $f_7 - f_8$ and f_{10}. The FEVs obtained by GPSO algorithm are much above "Threshold Error". In contrast, the OPSO algorithm was successful as the FEVs in these four benchmark functions remain below the "Threshold Error". Similar observations were made for the remaining benchmark functions.

The above mentioned observations provide the evidence of superior performance of the OPSO algorithm in terms of three fitness values, σ, convergence, SR and RR.

6.6 Sensitivity analysis of the OPSO algorithm against swarm's Size

In order to study the sensitivity analysis of the proposed OPSO algorithm against the changing in the swarm population, m, four benchmarks functions f_1, $f_7 - f_8$ and f_{10} are selected with a fixed number of iteration, i.e., N_{iter} = 1,000. The test is carried out with $N_{\mathrm{run}} = 20$. In addition, the AET is obtained at t =3,750 over 20 runs. Table 17 shows sensitivity analysis of the OPSO algorithm under effect of changing swarm's size m, from 40 to 100, on the performance of OPSO algorithm in terms of BFV, WFV, MFV and AET for f_1, $f_7 - f_8$ and f_{10}. We can see from Table 17 that the AET increases as m increases. This gives rise to more computation. In addition, it does not yield any tangible change or better solution in BFV, WFV and MFV. This result leads to that when $m >> d$, it gives rise to more computation, but does not provides any better solution. Considering these two extremes a reasonable value of m is could be about 10% more than d.

6.7 Comparison between the proposed OPSO algorithm and other ECTs

Here, we verify the performance of the proposed OPSO algorithm by comparing it with few ECTs recently reported by other authors [33-37], [48-52].

6.7.1 Shifted and rotated benchmark functions f_1, f_2, f_4 and f_6.

The three top-ranked algorithms in the CEC 2015 [31] learning based papers (LBP) are SPS-L-SHADE-EIG [48], DEsPA [49], and MVMO [50], in which the four shifted and rotated benchmark functions f_1, f_2, f_4 and f_6 are considered. Here, we compare the performance of the OPSO algorithm against the above three top-ranked algorithms and a few competitive algorithms from [33] for the four functions.

The comparison has been achieved with N_{run} = 20 and for each run the maximum NFE =10,000×d, i.e., 10,000×30 = 300,000, as given in [33]. The criterion used in this comparison depends on the values of maximum NFE and MFEV (41). When the algorithm reaches NFE = 300,000, the MFEV is recorded as a better result. The algorithm obtains a best result when the MFEV is 0.0 or close to 0.0.

Table 18 presents the results of the MFEV and the corresponding σ obtained by the eight ECTs. We can find that the OPSO algorithm is significantly superior to LLUDE, SaDE, JADE and CoDE [33] in solving f_1, f_2, f_4 and f_6. While comparing with CEC 2015-LBP [31] algorithms, performance of the OPSO algorithm found similar to that of the three top-ranked algorithms for the functions f_1 and f_2. However, the OPSO algorithm outperforms the three top-ranked algorithms for the function f_4 and f_6.

6.7.2 Shifted and rotated benchmark functions f_3, f_5, $f_7 – f_{10}$.

The benchmark functions in CEC 2015 expensive optimization papers (EOP) [32] are highly competitive and require efficient optimization algorithms to provide fast solutions with a high accuracy. The two top-ranked algorithms are MVMO [51] and TunedCMAES [52], in which the six shifted and rotated benchmark functions f_3, f_5, $f_7 – f_{10}$ are considered.

Here, we compare the performance of the OPSO algorithm with that of the above two top-ranked algorithms and few other competitive algorithms from [34-37]. The comparison has been achieved with N_{run} = 20 and for each run the exact maximum NFE =1,500 as given in [32]. The dimension of each tested function is d = 30. The 50 particles have been used in the DD-SRPSO [35] and SHPSO-GSA [37] algorithms, whereas 60 particles are used for the EPSO [36] algorithm. In this experiment, The OPSO algorithm uses 50 particles. Thus, the N_{iter} = 30 based on (40).

Table 19 shows the MFEV, the corresponding σ and AET of the seven ECTs. Among the seven ECTs, the OPSO algorithm achieves the best MFEV performance for the five functions, f_5, $f_7 – f_{10}$, whereas the MVMO [51] gives the best MFEV performance for the function f_3. In terms of σ, the performance of the OPSO algorithm is the best in case of the four functions f_5, $f_7 – f_9$ and is the second best for the functions, f_3 and f_{10}. Thus, the performance of the OPSO algorithm found to be superior to the two CEC 2015-EOP [32] algorithms. In terms of AET, the OPSO algorithm performance is better than SabDE [34] algorithm for all the functions except f_5.

6.8 Statistical significance of the proposed OPSO algorithm

In order to determine the statistical significance of the proposed OPSO algorithm, we carried out three sets of unpaired one-tailed t-Test [68] with a significance level of $\alpha = 0.05$. The results of the t-Test for CEC 2015-LBP [31] are shown in Table 20. Here, we provide the statistical results of the comparison between OPSO algorithm and seven competitive algorithms for f_1, f_2, f_4 and f_6. The comparison is made with a degree of freedom equals to 19. The OPSO algorithm is considered to be statistically significant against the contender algorithm when t-value < 0 and p-value less than 0.05. The general merit over contender is shown in the last row of Table 20. It is calculated as the difference between the number of times the OPSO algorithm is found to be statistically significant and statistically not significant among four tested functions. It can be seen that out of the seven algorithms, the OPSO algorithm is statistically significant against four algorithms, i.e., LLUDE, SaDE, JADE, and CoDE [33]. However, against the three top-ranked algorithms CEC 2015-LBP [31], SPS-L-SHADE-EIG [48], DEsPA [49], and MVMO [50], the OPSO algorithm is statistically significant for f_4 and f_6, whereas it is statistically not significant for f_1 and f_2.

Table 21 shows the statistical results of the comparison between OPSO algorithm and six competitive algorithms for the six functions, f_3, f_5, $f_7 - f_{10}$. The comparison is made with a degree of freedom equals to 19. The general merit over contender is shown in the last row of Table 21. It can be seen that the OPSO algorithm is statistically significant against all the six algorithms.

Table 22 shows the t-Test results for the three power systems, PS-1, PS-2, and PS-3. The one-tailed unpaired with $\alpha = 0.05$ with a degree of freedom of 99 is performed against twenty four competitive algorithms. As seen from the data in the last column, the proposed OPSO algorithm is found to be statistically significant against the first thirteen competitive algorithms for three power systems.

Since the data are not available for PS-1, PS-2, or PS-3 for eleven algorithms with Sl.No. 14 to 24, the t-Test is carried out against one or two power systems. Again, from the data in the last column, one can observe that the OPSO algorithm is statistically significant against the eleven contending algorithms. These results give enough evidence that the proposed OPSO algorithm is statistically significant against the twenty four contending algorithms.

7. Conclusion

A novel optimization algorithm named orthogonal PSO algorithm is proposed to alleviate the problems associated with the global PSO algorithm. An orthogonal diagonalization process is carried out in the OPSO algorithm which aims to diagonalize the position vectors of the active group particles. In contrast to two guides as used in GPSO algorithm, the OPSO algorithm uses only one guide while updating of the position and velocity vectors. The OPSO algorithm is applied for solving economic dispatch problem of thermal generating units (TGUs) under various practical power constraints imposed by the smart grid and power systems. The OPSO algorithm is tested with three practical power systems and ten selected CEC 2015 benchmark functions of increasing complexity and its superiority over GPSO algorithm and several existing ECTs has been shown with extensive simulation studies. The proposed OPSO algorithm has shown evidence of superior performance compared to several existing ECTs in providing reliable, consistent and optimal solution for the economic dispatch problem and shifted rotated CEC 2015 benchmark functions. The OPSO algorithm is found to be statistically significant against several ECTs including top-ranked CEC 2015 algorithms.

Appendix

$B = 1\times10^{-06}$

[0.6723	0.0915	0.5294	0.9081	0.7637	0.1982	0.0086	0.1108	0.8954	0.5386	0.7449	0.2905	0.6198	0.7461	0.1080	0.0560	0.2790	0.3400	0.2672	0.0919	0.8169	0.0832	0.7715	0.9262	0.8964	0.0499	0.5824	0.9758	0.8564	0.8281	0.4880	0.0137	0.9521	0.9316	0.3839	0.3796	0.5480	0.3747	0.7290	0.1868
0.4315	0.6146	0.8300	0.5522	0.5588	0.1951	0.7271	0.4075	0.5825	0.9917	0.8923	0.4026	0.2606	0.6625	0.4599	0.8169	0.6754	0.8467	0.7537	0.4021	0.1895	0.4617	0.7648	0.2985	0.4822	0.5459	0.4210	0.5554	0.8998	0.5074	0.4978	0.3741	0.9759	0.7831	0.2602	0.3191	0.5669	0.4369	0.2026	0.2472
0.6944	0.0110	0.8588	0.0329	0.1838	0.3268	0.3541	0.8841	0.5827	0.7552	0.2426	0.8621	0.4457	0.5233	0.4509	0.5289	0.9037	0.2461	0.8984	0.2952	0.1237	0.0304	0.4211	0.3381	0.0141	0.9432	0.0921	0.8463	0.2179	0.3662	0.9360	0.9227	0.0309	0.6857	0.8775	0.9861	0.6804	0.3043	0.2163	0.0542
0.2568	0.5733	0.7890	0.0539	0.4979	0.8803	0.7804	0.5481	0.8549	0.9805	0.1296	0.6147	0.8440	0.2599	0.5511	0.6944	0.9085	0.5815	0.7284	0.3065	0.8210	0.7532	0.0568	0.8595	0.6229	0.3215	0.0240	0.4081	0.0770	0.2266	0.3893	0.5465	0.4939	0.4662	0.8061	0.7182	0.3714	0.2909	0.9763	0.6090
0.0098	0.7897	0.3178	0.8051	0.5178	0.4711	0.4367	0.3690	0.0349	0.2348	0.2251	0.9912	0.1962	0.9620	0.8054	0.2124	0.7472	0.9377	0.4068	0.1056	0.6379	0.7000	0.5857	0.3405	0.2311	0.8065	0.4911	0.4620	0.4742	0.5348	0.1171	0.4739	0.8627	0.2603	0.4611	0.4132	0.0782	0.2425	0.5932	0.7772
0.5323	0.2354	0.4522	0.4514	0.9942	0.4040	0.4366	0.2083	0.8854	0.5286	0.3500	0.2037	0.3039	0.5402	0.7009	0.5433	0.2605	0.0478	0.9383	0.5938	0.0161	0.2145	0.1742	0.1381	0.5274	0.6014	0.2783	0.8263	0.8350	0.2895	0.2404	0.4965	0.2429	0.5693	0.0910	0.0986	0.4564	0.9367	0.3044	0.5111
0.2794	0.4480	0.7522	0.3826	0.8549	0.1792	0.0492	0.4409	0.4077	0.0514	0.2871	0.8272	0.4833	0.0303	0.8722	0.7025	0.6896	0.0540	0.2554	0.2827	0.8960	0.6799	0.7286	0.5078	0.7250	0.7896	0.3398	0.9912	0.4694	0.0684	0.6849	0.3090	0.8343	0.2488	0.5643	0.7346	0.0478	0.8602	0.9677	0.0278
0.9462	0.5694	0.1099	0.7896	0.9624	0.9689	0.0496	0.9562	0.0364	0.7569	0.9275	0.6759	0.3378	0.6963	0.0522	0.9564	0.1318	0.0206	0.5332	0.1552	0.5154	0.5573	0.5343	0.8567	0.6074	0.7992	0.2873	0.5239	0.4138	0.0850	0.8393	0.9508	0.8136	0.3193	0.1874	0.6373	0.7383	0.3972	0.8960	0.9904
0.9064	0.0614	0.1097	0.3643	0.6789	0.4075	0.0911	0.1240	0.7461	0.6020	0.0513	0.2489	0.7985	0.5197	0.2197	0.4445	0.1235	0.6815	0.9548	0.0007	0.5445	0.8507	0.2531	0.3843	0.5884	0.0496	0.1709	0.9254	0.5027	0.0683	0.9701	0.9820	0.6290	0.9108	0.5317	0.0738	0.0380	0.4794	0.1900	0.5009
0.3927	0.4963	0.2699	0.5323	0.4035	0.8445	0.5940	0.4708	0.1548	0.8572	0.5927	0.4758	0.9875	0.0590	0.4596	0.0854	0.1909	0.5986	0.2677	0.2836	0.6064	0.5586	0.9171	0.6957	0.4334	0.2832	0.3993	0.7390	0.1254	0.4098	0.2152	0.5136	0.0022	0.8852	0.3550	0.1205	0.9542	0.5650	0.0018	0.3320
0.0249	0.6423	0.5246	0.7117	0.9350	0.6153	0.2411	0.8569	0.1439	0.9883	0.1629	0.3991	0.1590	0.8900	0.9585	0.0573	0.1457	0.1140	0.2501	0.5508	0.7604	0.9018	0.7582	0.6279	0.2442	0.6535	0.6976	0.5674	0.1323	0.1234	0.7603	0.9926	0.3797	0.7946	0.3148	0.9816	0.7424	0.4896	0.7118	0.1739
0.6714	0.2213	0.9727	0.8715	0.4795	0.3766	0.8414	0.0434	0.6060	0.9295	0.8384	0.5994	0.2369	0.3302	0.7900	0.6295	0.5850	0.7962	0.9277	0.8709	0.8553	0.4195	0.8870	0.4504	0.4290	0.4897	0.2037	0.9688	0.8705	0.4430	0.5841	0.4558	0.9044	0.9258	0.7267	0.4968	0.9374	0.2698	0.8677	0.6256
0.8372	0.8371	0.7104	0.3287	0.2318	0.8772	0.8572	0.6916	0.2545	0.4095	0.1676	0.8005	0.7022	0.2297	0.4519	0.7962	0.0734	0.6179	0.0686	0.0423	0.3829	0.3581	0.0688	0.4736	0.0102	0.9729	0.6663	0.8245	0.6030	0.8989	0.4030	0.4260	0.6804	0.1788	0.5158	0.0224	0.5134	0.9897	0.1183	0.5751
0.9715	0.9711	0.3119	0.6501	0.3963	0.7849	0.9636	0.9790	0.3242	0.0003	0.5022	0.1051	0.3755	0.1139	0.3334	0.6912	0.8223	0.0702	0.2994	0.9047	0.0846	0.4890	0.1835	0.9497	0.6088	0.7485	0.4431	0.9596	0.2653	0.3536	0.5100	0.2132	0.3788	0.5175	0.7906	0.0538	0.2409	0.1837	0.0390	0.7510
0.0569	0.8464	0.2915	0.9748	0.7051	0.4650	0.4889	0.2833	0.4018	0.5409	0.9993	0.8214	0.9737	0.3109	0.0591	0.3453	0.7229	0.0693	0.5916	0.1310	0.7339	0.2560	0.7371	0.0835	0.9580	0.5678	0.4333	0.6463	0.8648	0.1202	0.4956	0.1932	0.6320	0.6270	0.2045	0.1409	0.2600	0.8617	0.5982	0.1535
0.4503	0.5060	0.8504	0.0760	0.5586	0.8140	0.2203	0.1338	0.4064	0.2077	0.3554	0.8411	0.9723	0.2284	0.7409	0.9468	0.9259	0.1360	0.2033	0.8337	0.3320	0.9292	0.6967	0.2798	0.0356	0.2561	0.1932	0.4766	0.4578	0.8750	0.7437	0.7266	0.5714	0.6640	0.0525	0.4658	0.7590	0.0326	0.6043	0.3568
0.5825	0.2789	0.9116	0.5870	0.7566	0.8984	0.2262	0.6853	0.3862	0.2193	0.0471	0.3545	0.6437	0.6520	0.5068	0.5202	0.4926	0.7889	0.6359	0.8005	0.8397	0.4668	0.7770	0.4470	0.8862	0.8866	0.6164	0.9119	0.7222	0.3486	0.3020	0.5297	0.9817	0.3892	0.8012	0.5609	0.9933	0.3320	0.5164	0.1440
0.6866	0.7466	0.6393	0.4139	0.9955	0.4292	0.5368	0.9095	0.6098	0.3258	0.2137	0.4301	0.8601	0.0662	0.1999	0.9538	0.6549	0.0924	0.7984	0.9179	0.3717	0.2540	0.5019	0.5876	0.2469	0.4468	0.2690	0.0149	0.3390	0.0419	0.0896	0.8291	0.8497	0.7400	0.6786	0.4945	0.3567	0.7487	0.0075	0.8506
0.7194	0.2369	0.2554	0.3091	0.9624	0.3343	0.7621	0.6109	0.1669	0.0959	0.3978	0.5722	0.4019	0.2754	0.4272	0.0736	0.8901	0.2379	0.5017	0.1373	0.8282	0.4312	0.4255	0.8776	0.0089	0.8160	0.5597	0.1567	0.4012	0.1423	0.8260	0.5119	0.2834	0.8176	0.9460	0.0678	0.7529	0.6444	0.6889	0.3379
0.6500	0.9573	0.0887	0.2638	0.5351	0.5966	0.3476	0.9000	0.1881	0.7475	0.3337	0.7008	0.6319	0.2818	0.1687	0.2070	0.5385	0.2436	0.6508	0.5047	0.1765	0.7025	0.6112	0.4691	0.8149	0.0983	0.9448	0.4716	0.5270	0.0766	0.3896	0.5520	0.6825	0.6003	0.0916	0.8976	0.1100	0.1692	0.9460	0.2752
0.7269	0.6203	0.8383	0.7588	0.9639	0.9020	0.4612	0.1934	0.0946	0.7485	0.2296	0.7425	0.9852	0.8801	0.7517	0.7750	0.2822	0.1048	0.7960	0.4050	0.1295	0.4023	0.8558	0.4374	0.1405	0.8596	0.7145	0.5430	0.8942	0.7405	0.7753	0.2133	0.3581	0.0850	0.9084	0.2886	0.5970	0.9522	0.8735	0.0060
0.3738	0.6003	0.5847	0.9952	0.1156	0.7021	0.6393	0.7544	0.3232	0.5433	0.9361	0.7579	0.5595	0.4443	0.3684	0.9142	0.9760	0.8584	0.2334	0.1736	0.8799	0.1818	0.6708	0.7462	0.8799	0.0276	0.6792	0.0597	0.7784	0.4565	0.1794	0.5878	0.9869	0.9224	0.5100	0.2690	0.4306	0.5433	0.1133	0.8019
0.5816	0.1726	0.9481	0.1866	0.0514	0.3775	0.9173	0.3463	0.7696	0.3381	0.6832	0.3891	0.9336	0.7559	0.9418	0.7826	0.0364	0.6982	0.6008	0.5752	0.0441	0.8563	0.5236	0.4679	0.0954	0.8992	0.9594	0.6580	0.0694	0.6682	0.1094	0.1428	0.0840	0.0536	0.6149	0.5942	0.7307	0.2514	0.3546	0.4974
0.1161	0.0903	0.0610	0.7811	0.3043	0.7350	0.1616	0.4186	0.2341	0.8323	0.9621	0.4293	0.7203	0.6033	0.0172	0.2955	0.3262	0.7337	0.1125	0.6062	0.6867	0.5842	0.2988	0.8608	0.3526	0.8999	0.7753	0.8896	0.2788	0.6992	0.9052	0.0522	0.2503	0.5270	0.3161	0.4759	0.2612	0.5786	0.2419	0.5378
0.0577	0.2553	0.5846	0.1958	0.5802	0.9541	0.7156	0.1557	0.7404	0.5526	0.4380	0.9563	0.4840	0.7833	0.8291	0.1518	0.9730	0.6505	0.5158	0.2144	0.7338	0.3736	0.7040	0.4665	0.5934	0.5241	0.6077	0.1096	0.3794	0.5714	0.8764	0.6833	0.8113	0.1189	0.0775	0.3683	0.0948	0.9155	0.5603	0.8709
0.9798	0.8586	0.2851	0.9924	0.5310	0.5428	0.5777	0.8190	0.6928	0.9575	0.9403	0.5730	0.6390	0.1139	0.6266	0.8479	0.3650	0.5163	0.8378	0.5199	0.4372	0.2217	0.3816	0.4981	0.5852	0.1202	0.9480	0.4378	0.8647	0.6287	0.9998	0.6086	0.0844	0.3801	0.8506	0.6556	0.4510	0.8956	0.6127	0.7228
0.2848	0.9111	0.8277	0.8023	0.9012	0.5401	0.4333	0.6249	0.8241	0.8928	0.0058	0.8497	0.8876	0.9786	0.5387	0.7849	0.3091	0.3264	0.9208	0.9892	0.3798	0.2190	0.5677	0.4874	0.6677	0.1778	0.0596	0.2802	0.4200	0.8778	0.8643	0.2197	0.5313	0.8128	0.1445	0.9382	0.6401	0.4825	0.3008	0.6681
0.5950	0.6996	0.1910	0.4242	0.5406	0.3111	0.8842	0.7386	0.8280	0.3565	0.6103	0.2763	0.1987	0.8486	0.6505	0.2708	0.1209	0.6618	0.4982	0.4899	0.9797	0.5222	0.8879	0.2295	0.6480	0.7061	0.2687	0.9852	0.2399	0.6624	0.0369	0.4063	0.8006	0.2441	0.3705	0.6204	0.1320	0.4427	0.7981	0.1788
0.9622	0.7252	0.4425	0.7289	0.4320	0.0712	0.3931	0.8051	0.2934	0.5464	0.8011	0.6223	0.3954	0.0506	0.7266	0.2278	0.9158	0.1176	0.2776	0.6949	0.3990	0.4334	0.8429	0.0856	0.4334	0.8314	0.9867	0.6088	0.5977	0.8754	0.5447	0.6299	0.7388	0.8844	0.6224	0.2828	0.4528	0.3118	0.7956	0.5505
0.1858	0.2299	0.3934	0.4984	0.5427	0.1820	0.1790	0.0672	0.3094	0.3467	0.2330	0.5884	0.9922	0.4662	0.0945	0.3210	0.1355	0.1478	0.6525	0.4114	0.4402	0.7413	0.8988	0.0674	0.1398	0.0348	0.7722	0.2537	0.4794	0.4675	0.9976	0.5553	0.1417	0.7126	0.9976	0.2052	0.6522	0.0553	0.7811	0.9599
0.1930	0.5761	0.8266	0.8090	0.7124	0.0930	0.6333	0.9508	0.5230	0.6228	0.9325	0.9635	0.4024	0.3257	0.8776	0.8296	0.3321	0.0198	0.9173	0.0348	0.1568	0.0704	0.9390	0.8884	0.7519	0.7578	0.4754	0.1326	0.8985	0.1413	0.5110	0.1276	0.4379	0.3781	0.5173	0.4391	0.8270	0.7538	0.3511	0.5960
0.3416	0.8106	0.6769	0.3565	0.0167	0.4635	0.6240	0.4976	0.3253	0.7966	0.7633	0.0859	0.6589	0.6302	0.0144	0.8222	0.8975	0.9643	0.5098	0.2928	0.3260	0.8473	0.8154	0.2332	0.2418	0.9571	0.6809	0.5450	0.9347	0.0681	0.8735	0.1692	0.3504	0.2489	0.9905	0.0273	0.3081	0.1319	0.0543	0.8086
0.9329	0.4038	0.2076	0.0732	0.8009	0.0093	0.3279	0.7551	0.8318	0.7459	0.8264	0.5005	0.9013	0.2303	0.2943	0.5707	0.4996	0.9704	0.9742	0.8014	0.3141	0.6799	0.0014	0.8616	0.6505	0.3429	0.4169	0.8278	0.8179	0.7142	0.0702	0.0010	0.4785	0.2529	0.2265	0.8762	0.4024	0.3559	0.7087	0.9845
0.3907	0.9884	0.3181	0.5910	0.1425	0.9150	0.8030	0.7424	0.8103	0.1255	0.5735	0.5216	0.9954	0.5799	0.1799	0.5718	0.6153	0.1239	0.1973	0.3465	0.8945	0.1367	0.0031	0.7117	0.8574	0.6382	0.3801	0.8370	0.7089	0.3080	0.9875	0.4182	0.5874	0.7672	0.3980	0.6101	0.8842	0.3959	0.9929	0.8859
0.2732	0.0900	0.1338	0.9102	0.4785	0.6427	0.9995	0.8311	0.5570	0.8224	0.7926	0.0902	0.6532	0.6032	0.9263	0.2860	0.5831	0.4674	0.1112	0.0833	0.2470	0.8584	0.0875	0.8728	0.0844	0.3430	0.2133	0.8333	0.7432	0.6712	0.9227	0.4885	0.1458	0.0499	0.6966	0.2036	0.7006	0.8855	0.1625	0.2138
0.1519	0.3209	0.6715	0.1938	0.2568	0.0014	0.9810	0.1565	0.2630	0.0252	0.3290	0.9047	0.1084	0.5999	0.0682	0.6991	0.6983	0.6567	0.2974	0.5111	0.3107	0.1998	0.2607	0.9380	0.9721	0.2165	0.3829	0.2037	0.8997	0.6524	0.5643	0.1599	0.9053	0.6853	0.0646	0.5199	0.2419	0.0212	0.1136	0.0346
0.3971	0.5114	0.5710	0.4324	0.3691	0.0304	0.1270	0.4573	0.6806	0.4144	0.2235	0.8844	0.0361	0.4484	0.5811	0.7963	0.0293	0.2902	0.3964	0.3668	0.4089	0.6073	0.0228	0.1397	0.0315	0.7862	0.0297	0.5444	0.0652	0.5310	0.4315	0.6668	0.6402	0.6203	0.7477	0.0538	0.7598	0.8441	0.9129	0.4511
0.3747	0.0606	0.1698	0.7492	0.6618	0.2085	0.2322	0.6181	0.2337	0.7314	0.3124	0.4390	0.6181	0.0354	0.6372	0.4416	0.5279	0.7545	0.4208	0.7395	0.7080	0.5430	0.4241	0.3939	0.8354	0.7231	0.4723	0.8749	0.3359	0.7151	0.3378	0.0179	0.1629	0.7467	0.4204	0.8622	0.2909	0.2881	0.4817	0.0138
0.1311	0.7257	0.1477	0.0392	0.1696	0.4550	0.0236	0.9322	0.4564	0.7814	0.5845	0.7817	0.5671	0.5138	0.6513	0.4462	0.0321	0.5581	0.3115	0.5247	0.1436	0.1623	0.3411	0.9806	0.8357	0.2788	0.3334	0.1210	0.0043	0.5048	0.7207	0.1197	0.5659	0.9773	0.8113	0.4429	0.2774	0.2503	0.8518	0.4737
0.4350	0.5566	0.4761	0.9463	0.2788	0.1273	0.6074	0.8351	0.3846	0.3673	0.8299	0.1480	0.9620	0.4077	0.8646	0.4657	0.8271	0.4278	0.6938	0.8045	0.8713	0.0057	0.5414	0.6448	0.4350	0.5566	0.4761	0.9463	0.2788	0.1273	0.6074	0.8351	0.3846	0.3673	0.8299	0.1480	0.0061	0.4884	0.8099	0.9512]

References

[1] Y. Xu, W.Zhang, W. Liu, Distributed dynamic programming-based approach for economic dispatch in smart grid, IEEE Transaction on Industrial Informatics 11(1) (2015) 166-175.

[2] Y. Zhang, D. Gong, N. Geng, X. Sun, Hybrid bare-bones PSO for dynamic economic dispatch with valve-point effects, Applied Soft Computing 18 (2014) 248-260.

[3] KT. Chatuverdi, M. Pandit, L. Srivastava, Self-organizing hierarchical particle swarm optimization for nonconvex economic dispatch, IEEE Transaction on Power Systems 23 (2008) 1079-1087.

[4] F. Meng, X. Chen, Y. Zhang, Consistency-based linear programming models for generating the priority vector from interval fuzzy preference relations, Applied Soft Computing 41 (2016) 247-264.

[5] A. Liefooghe, S. Verel, J-K, Hao, A hybrid metaheuristic for multiobjective unconstrained binary quadratic programming, Applied Soft Computing 16 (2014) 10-19.

[6] V. Sharma, R. Jha, R. Naresh, Optimal multi-reservoir network control by augmented Lagrange Programming neural network, Applied Soft Computing 7 (2007) 783-790.

[7] A. Theerthamalai, S. Maheswarapu, An effective non-iterative "λ-logic based" algorithm for economic dispatch of generators with cubic fuel cost function, International Journal of Electrical and Power Energy System 32 (2010) 539-542.

[8] A. heydari, S.N. Balakrishnan, Global optimality of approximate dynamic programming and its use in non-convex function function minimization, Applied Soft Computing 24 (2014) 291-303.

[9] P. Chen, H. Chang, Large-scale economic dispatch by genetic algorithm, IEEE Transactions on Power Systems10 (4) (1995) 1919-1926.

[10] H. Zhang, D. Yue, X. Xie, S. Hu, S. Weng, Multi-elite guide hybrid differential evolution with simulated annealing technique for dynamic economic emission dispatch, Applied Soft Compting 34 (2015) 312-323.

[11] P. Savsani, R.L. Jhala, V. Savsani, Effect of hybridizing biogeography-based optimization (BBO) technique with artificial immune algorithm (AIA) and ant colony optimization (ACO), Applied Soft Computing 21 (2014) 542-553.

[12] D. Aydin, S. Özyön, Solution to non-convex economic dispatch problem with valve point effects by incremental artificial bee colony with local search, Applied Soft Computing 13 (2013) 2456-2466

[13] V. Dieu, P. Schegner, Augmented Lagrange Hopfield network initialized by quadratic programming for economic dispatch with piecewise quadratic cost functions and prohibited zones, Applied Soft Computing 13 (2013) 292-301.

[14] A. Bhattacharya, P. Chattopadhaay, Solving economic dispatch load dispatch problems using hybrid differential evolution, Applied Soft Computing 11(2011)2526-2537.

[15] M. Al-Betar, M. Awadallah, A. Khader, A. Bolaji, Tournament-based harmony search algorithm for non-convex economic dispatch problem, Apllied Soft Computing 47 (2016) 449-459.

[16] J. Kennedy, C. Eberhart, Particle swarm optimization, in: Proceedings of the IEEE International Conference on Neural Networks, 1995 (4), pp. 1942-1948.

[17] R. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, in: Proceedings of the sixth International Symposium on Micro Machine and Human Science, 1995, pp. 39-43.

[18] Z. Zhan, J. Zhang, Y. Li, Y. Shi, Orthogonal learning particle swarm optimization, IEEE Transactions on Evolutionary Computation 6 (2011) 832-847.

[19] W. Gao, S. Liu, L. Huang, A novel artificial bee colony algorithm based modified search equation and orthogonal learning, IEEE Transactions on Cybernetics 43(3) (2013) 1011-1024.

[20] Y. Leung, Y. Wang, An orthogonal genetic algorithm with quantization for global numerical optimization, IEEE Transactions on Evolutionary Computation 5 (1) (2011) 41-53.

[21] M. Basu, Modified particle swarm optimization for nonconvex economic dispatch problems. International Journal of Electrical Power and Energy Systems 69 (2015) 304-312.

[22] P. Subbaraj, R. Rengaraj, S. Salivahanan, T. Senthilkumar, Parallel particle swarm optimization with modified stochastic acceleration factors for solving large scale economic dispatch problem. International Journal of Electrical Power and Energy Systems 32 (2010) 1014-1023.

[23] J. Park, Y. Jeong, J. Shin, An Improved particle swarm optimization for nonconvex economic dispatch problems, IEEE Transactions on Power Systems 25 (1) (2010) 156-166.

[24] S.H. Ling, C. Iu, K. Chan, K. Lam, B, Yeung, F, Leung, Hybrid particle swarm optimization with wavelet mutation and its industrial applications. IEEE Transactions on Systems, Man, and Cybernetics 38 (3) 2008 743–63.

[25] J. Sun, V. Palade, X. Wu, W. Fang, Z. Wang, Solving the power economic dispatch problem with generator constraints by random drift particle swarm optimization, IEEE Transactions on Power Systems 13 (1) (2014) 519–526.

[26] C. Kuo, A Novel coding scheme for practical economic dispatch by modified particle swarm approach, IEEE Transactions on Power Systems 23 (4) (2008) 1825-1835.

[27] I. Selvakumar, K. Thanushkodi, A new particle swarm optimization solution to nonconvex economic dispatch problems, IEEE Transactions on Power Systems 22 (1) (2007) 42-51.

[28] A. Selvakumar, K. Thanushkodi, Anti-predatory particle swarm optimization: solution to nonconvex economic dispatch problems, International Journal of Electric Power Systems Research 78 (2008) 2-10.

[29] J. Cai, X. Ma, L. Li, H. Peng, Chaotic particle swarm optimization for economic dispatch considering the generator constraints, Energy Conversation and Management 48 (2007) 645-653.

[30] S. Chakraborty, T. Senjyu, A. Saber, A. Yona, T. Funabashi, A Novel Particle Swarm Optimization Method Based on Quantum Mechanics Computation for Thermal Economic Load Dispatch Problem, IEEJ Transactions on Electrical and Electronic Engineering 7 (2012) 461-470.

[31] J.J. Liang, B.Y. Qu, P.N. Suganthan, Q. Chen, Problem definitions and evaluation criteria for the CEC 2015 competition on learning-based real-parameter single objective optimization, Zhengzhou University/Nanyang Technological University, Zhengzhou, China/Singapore, 2014, Technical Report.

[32] Q. Chen, B. Liu, Q. Zhang, J.J. Liang, P.N. Suganthan, B.Y. Qu, Problem definition and evaluation criteria for CEC 2015 special session and competition on bound constrained single-

objective computationally expensive numerical optimization, in: IEEE C. Evol. Comput, Sendai, Japan, 2014, Special session.

[33] X.-G. Zhou, G.-J. Zhan, X.-H. Hao, D.-W. Xu, L. Yu, Enhance differential evolution using local Lipschits underestimate strategy for computationally expensive optimization problems, Applied Soft Computing 48 (2016) 169-181.

[34] A. Banitalebi, M. Aziz, Z. Aziz, a self–adaptive binary differential evolution algorithm for large scale binary optimization problems, Information Sciences 367-368 (2016) 487-511.

[35] M. Tanweer, R. Auditya, S. Suresh, N. Sundararajan, N. Srikanth, Directionally driven self-regulating particle swarm optimization algorithm, Swarm and Evolutionary Computation 28 (2016) 98-116.

[36] T. Ngo, A. Sadollah, J. Kim, A cooperative particle swarm optimizer with stochastic movements for computationally expensive numerical optimization problems, Journal of Computational Science 13 (2016) 68-82.

[37] A. Yadav, K. Deep, J. Kim, A. Nagar, Gravitational swarm optimizer for global optimization, Swarm and Evolutionary Computation, ARTICLE IN PRESS.

[38] W. Elsayed, E. El-Saadany, A fully decentralized approach for solving the economic dispatch problem, IEEE Transactions on Power Systems 30 (4) (2015) 2179-2189.

[39] A. Bhattacharya, P. Chattopadhyay, Biogeography-based optimization for different economic load dispatch problems, IEEE Transactions on Power Systems 25 (2) 2010 1064-1077.

[40] I. Ciornei, E. Kyriakide, A GA-API solution for the economic dispatch of generation in power system operation, IEEE Transactions on Power Systems 27 (1) (2012) 233-242.

[41] T. Ding, R. Bo, F. Li, H. Sun, A bi-level branch and bound method for economic dispatch with disjoint prohibited zones considering network losses, IEEE Transactions on Power System 30 (6) 2015 2841-2855.

[42] R. Azizipanah-Abarghooee, T. Niknam, M. Gharibzadeh, F. Golestaneh, Robust, fast and optimal solution of practical economic dispatch by a new enhanced gradient-based simplified swarm optimisation algorithm, IET Generation, Transmission & Distribution 7 (6) (2013) 620-35.

[43] T. Adhinarayanan, M. Sydulu, Efficient Lambda logic based optimization procedure to solve the large scale generator constrained economic dispatch problem. International Journal of Electrical Engineering and Technology 4 (3) (2009) 301-309.

[44] S. Wang, J. Chiou, C. Liu, Non-smooth/non-convex economic dispatch by a novel hybrid differential evolution algorithm, IET Generation, Transmission & Distribution 1 (5) (2007) 793-703.

[45] R. Naresh, J. Dubey, J. Sharma, Two-phase neural network based modelling framework of constrained economic load dispatch, IEE Generation, Transmission, Distribution 151(3) (2004) 373 -378.

[46] N. Singh, J. Dhillon, D. Kothari, Synergic predator-prey optimization for economic thermal power dispatch problem, Applied Soft Computing 43 (2016) 298-311.

[47] X. He, Y. Rao, J. Huang, A novel algorithm for economic load dispatch of power systems, Neurocomputing 171 (2016) 1454-1461.

[48] S.M. Guo, J.S.H. Tsai, C.C. Yang, P.H. Hsu, A self-optimization approach for L-SHADE incorporated with eigenvector-based crossover and successful-parent-selecting framework on CEC 2015 benchmark set, in: Proceedings of the 2015 IEEE Congress on Evolutionary Computation (CEC), 2015, pp. 1003-1010.

[49] N. Awad, M.Z. Ali, R.G. Reynolds, A differential evolution algorithm with success-based parameter adaptation for CEC2015 learning-based optimization, in: Proceedings of the 2015 IEEE Congress on Evolutionary Computation (CEC), 2015, pp. 1098-1105.

[50] J.L. Rueda, I. Erlich, Testing MVMO on learning-based real-parameter single objective benchmark optimization problems, in: Proceedings of the 2015 IEEE Congress on Evolutionary Computation (CEC), 2015, pp. 1025-1032.

[51] J.L. Rueda, I. Erlich, MVMO for bound constrained single-objective computationally expensive numerical optimization, in: Proceedings of the 2015 IEEE Congress on Evolutionary Computation (CEC), 2015, pp. 1011-1017.

[52] V. Berthier, Experiments on the CEC 2015 expensive optimization testbed, in: Proceedings of the 2015 IEEE Congress on Evolutionary Computation (CEC), 2015, pp. 1059-1066.

[53] L.T. Al Bahrani, J.C. Patra, Orthogonal PSO algorithm for solving ramp rate constraints and prohibited operating zones in smart grid applications, in: Proceedings of the International Joint Conference on Neural Networks (IJCNN) Killarney, Ireland, 2015, pp. 1-7.

[54] L.T. Al Bahrani, J.C. Patra, Orthogonal PSO algorithm for economic dispatch of power under power grid constraints, in: Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC), Kowloon, Hong Kong, 2015, p. 14-19.

[55] L.T. Al Bahrani, J.C. Patra, R. Kowalczyk, Orthogonal PSO algorithm for optimal dispatch of power of large-scale thermal generrating units in smart power grid under power grid constraints, in: Proceedings of the International Joint Conference on Neural Networks (IJCNN), Vancouver, Canada, 2016, pp. 660-7.

[56] K.K Mandal, S. Mandal, B. Bhattacharya, N. Chakraborty, Non-convex emission constrained dispatch using a new self-adaptive particle swarm optimization technique, Applied Soft Computing 28 (2015) 188-195.

[57] N. Pandit, A. Tripathi, S. Tapaswi, M.Pandit, An improved bacterial foraging algorithm for combined static/dynamic environmental economic dispatch, Applied Soft Computing 12 (2012) 3500-3513.

[58] D. Datta, Unit commitment problem with ramp rate constraint using a binary-real-coded genetic algorithm, Applied Soft Computing 13 (2013) 3873-3883.

[59] A.K. Barisal, R.C. Prusty, Large scale economic dispatch of power systems using oppositional invasive weed optimization, Applied Soft Computing 29 (2015) 122-137.

[60] P. Roy, S. Bhui, C,Paul, Solution of economic load dispatch using hybrid chemical reaction optimization approach, Applied Soft Computing 24 (2014) 109-125.

[61] Y.H. Shi, R.C. Eberhart, A modified particle swarm optimizer, in: Proceedings of the IEEE International Conference on Evolutionary Computation, 1998, pp. 69-73.

[62] R.C. Eberhart, Y.H. Shi, Comparing inertia weights and constriction factors in particle swarm optimization, in: Proceedings of the IEEE International Conference on Evolutionary Computation, 2000, pp. 84-88.

[63] D. Bratton, J. Kennedy, Defining a standard for particle swarm optimization, in: Proceedings of the 2007 IEEE Swarm Intelligence Symposium (SIS 2007), 2007, pp. 120-127.

[64] R. Poli, J. Kennedy, T. Blackwell, Particle swarm optimization, in: Proceedings of Swarm Intelligence 2007, p. 33–57.

[65] Z-H. Ruan, Y. Yuan, Q-X. Chen, C-X. Zhang, Y. Shuai, H-P Tan, A new multi-function global particle swarm optimization, Applied Soft Computing 49 (2016) 279-291.

[66] T. Niknam, H. Mojarrad, H. Meymand, Non-smooth economic dispatch computation by fuzzy and self adaptive particle swarm optimization, Applied Soft Computing 11 (2011) 2805-2817.

[67] S. Grossman, Elementary Linear Algebra, USA, Saunders College Publishing, 1994.

[68] D.J. Sheskin, Handbook of parametric and nonparametric statistical procedures, USA, Western Connecticut State University, 2000.

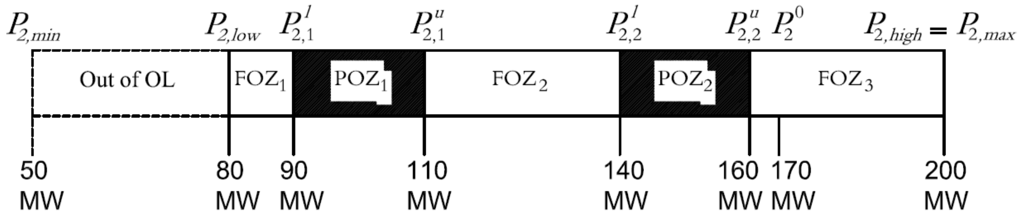


Fig. 1. Lower and upper generation limits, POZs and FOZs for TGU_2

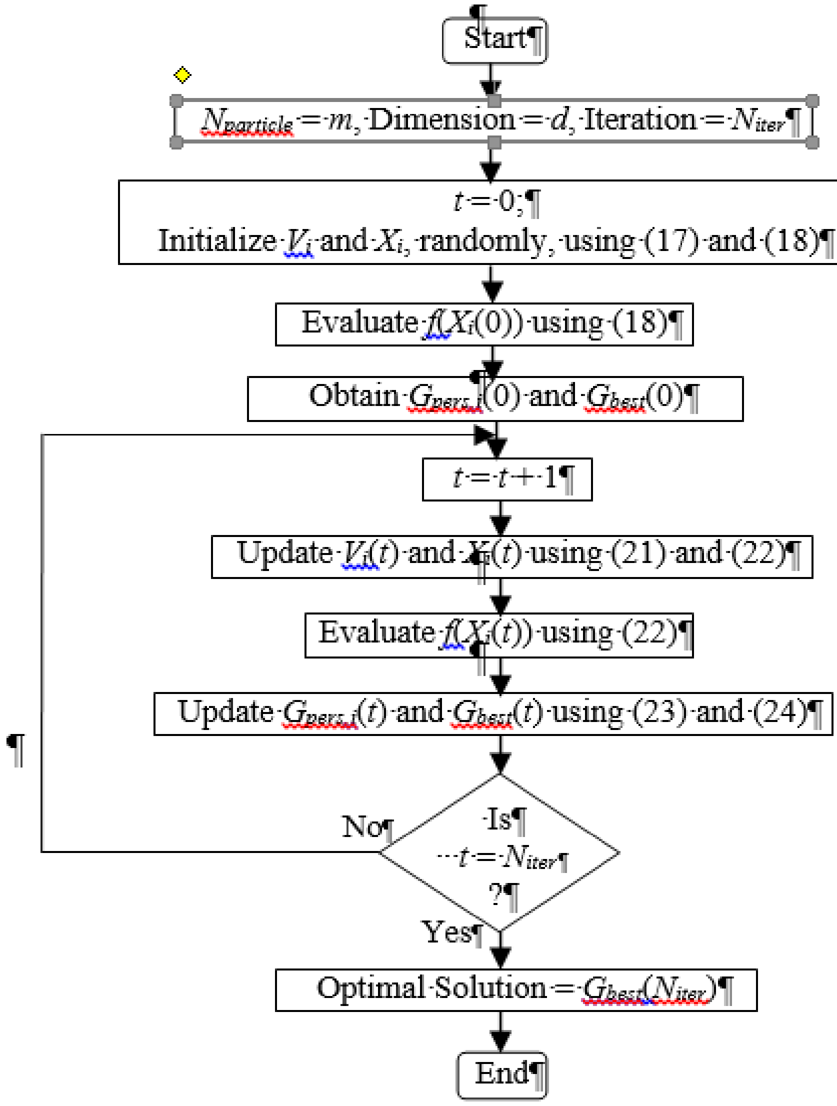


Fig. 2. Flowchart of the GPSO algorithm

L1: Let B be a real symmetric matrix of size $d \times d$.
L2: Apply Gram-Schmidt orthogonalization on matrix B to obtain d orthonormal vectors [67].
L3: Construct orthonormal matrix C using these vectors.
L4: Obtain the diagonal matrix D using (28).

Fig. 3. The orthogonal diagonalization process

```
Procedure: Convert matrix A( m×d) to a symmetric matrix B (d×d).

for i = 1 : d
    B(1, i) = A(1, i)
    B(i, 1) = A(1, i)
end for
for k = 2 : d
    for i = 2 : d
        B(k, i) = A(k, i)
        B1(k, i) = B(k, i)
        B(i, k) = B1(k, i)
    end for
end for
```

Fig. 4. Pseudocode for converting matrix *A* to a symmetric matrix *B*.

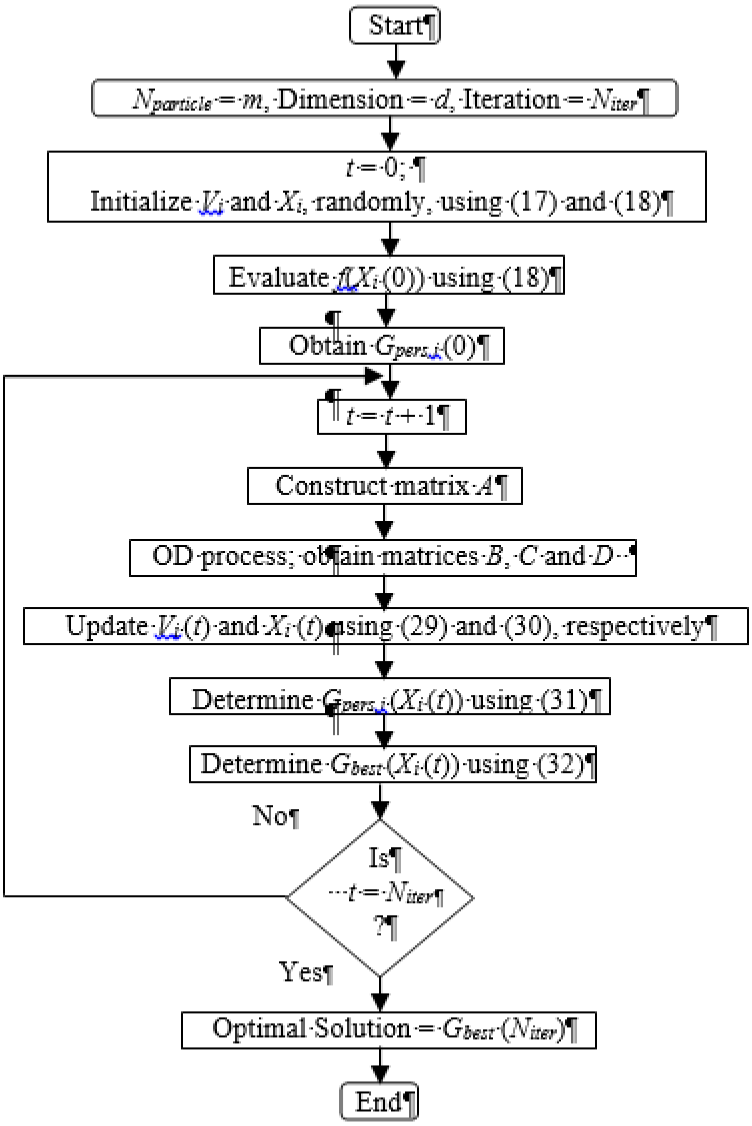


Fig. 5**. Flowchart of the OPSO algorithm**

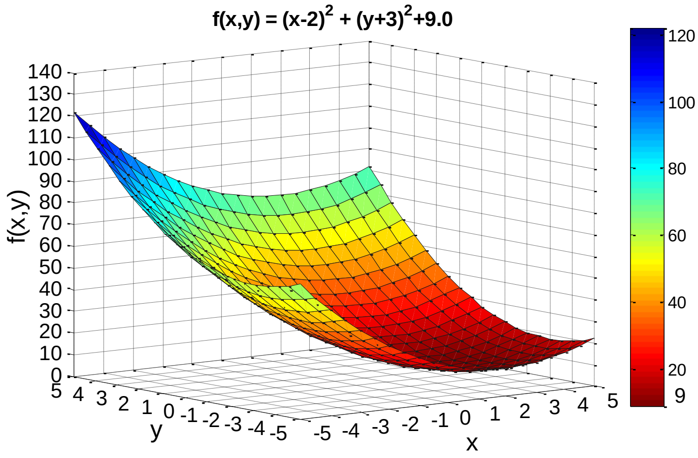


Fig. 6. Fitness landscape of $f(x,y)$. The minimum value of the function is 9.0 at $x = 2.0$ and $y = –3.0$.

***t* = 1**

$$D = \begin{bmatrix} \mathbf{-33.4275} & \mathbf{0.0000} \\ \mathbf{0.0000} & \mathbf{9.9113} \end{bmatrix}$$

$$X = \begin{bmatrix} \mathbf{55.4176} & \mathbf{-31.9043} & -80.94 & -23.13 & -6.20 & 79.95 \\ \mathbf{38.2285} & \mathbf{-4.3210} & 23.47 & 23.95 & -1.65 & 49.64 \end{bmatrix}$$

$$G_{pers} = \begin{bmatrix} \mathbf{-6.2957} & \mathbf{10.2996} & -23.13 & 55.41 & -80.94 & 85.23 \\ \mathbf{-1.6169} & \mathbf{23.4768} & 23.95 & 38.22 & 23.47 & -40.30 \end{bmatrix}$$

***t* = 80**

$$D = \begin{bmatrix} \mathbf{-3.7383} & \mathbf{0.0000} \\ \mathbf{0.0000} & \mathbf{3.3275} \end{bmatrix}$$

$$X = \begin{bmatrix} \mathbf{-28.0374} & \mathbf{6.7003} & 43.83 & -92.54 & 96.11 & -99.44 \\ \mathbf{-14.4267} & \mathbf{16.8004} & 94.80 & 63.09 & -90.95 & 49.64 \end{bmatrix}$$

$$G_{pers} = \begin{bmatrix} \mathbf{2.0074} & \mathbf{1.4628} & -6.20 & -23.13 & 0.81 & -80.94 \\ \mathbf{-2.7485} & \mathbf{-2.7543} & -1.65 & 23.95 & 49.64 & 23.47 \end{bmatrix}$$

***t* = 140**

$$D = \begin{bmatrix} \mathbf{-4.4026} & \mathbf{0.0000} \\ \mathbf{0.0000} & \mathbf{3.4027} \end{bmatrix}$$

$$X = \begin{bmatrix} \mathbf{-4.4068} & \mathbf{0.0001} & 99.98 & -92.54 & 96.11 & -99.44 \\ \mathbf{-0.0002} & \mathbf{3.4019} & 94.80 & 63.09 & -90.95 & 49.64 \end{bmatrix}$$

$$G_{pers} = \begin{bmatrix} \mathbf{1.9953} & \mathbf{1.9962} & -6.20 & -23.13 & 0.81 & -80.94 \\ \mathbf{-3.0007} & \mathbf{-2.9972} & -1.65 & 23.95 & 49.64 & 23.47 \end{bmatrix}$$

***t* = 200**

$$D = \begin{bmatrix} \mathbf{-4.4051} & \mathbf{0.0000} \\ \mathbf{0.0000} & \mathbf{3.4051} \end{bmatrix}$$

$$X = \begin{bmatrix} \mathbf{-4.4051} & \mathbf{0.0000} & 99.98 & -92.54 & 96.11 & -99.44 \\ \mathbf{0.0000} & \mathbf{3.4051} & 94.80 & 63.09 & -90.95 & 49.64 \end{bmatrix}$$

$$G_{pers} = \begin{bmatrix} \mathbf{2.0000} & \mathbf{2.0000} & -6.20 & -23.13 & 0.81 & -80.94 \\ \mathbf{-3.0000} & \mathbf{-3.0000} & -1.65 & 23.95 & 49.64 & 23.47 \end{bmatrix}$$

$$\underbrace{\textbf{\textit{Active Group}}}\qquad\underbrace{\textit{Passive Group}}$$

Fig. 7. A numerical example showing convergence of the OPSO algorithm.

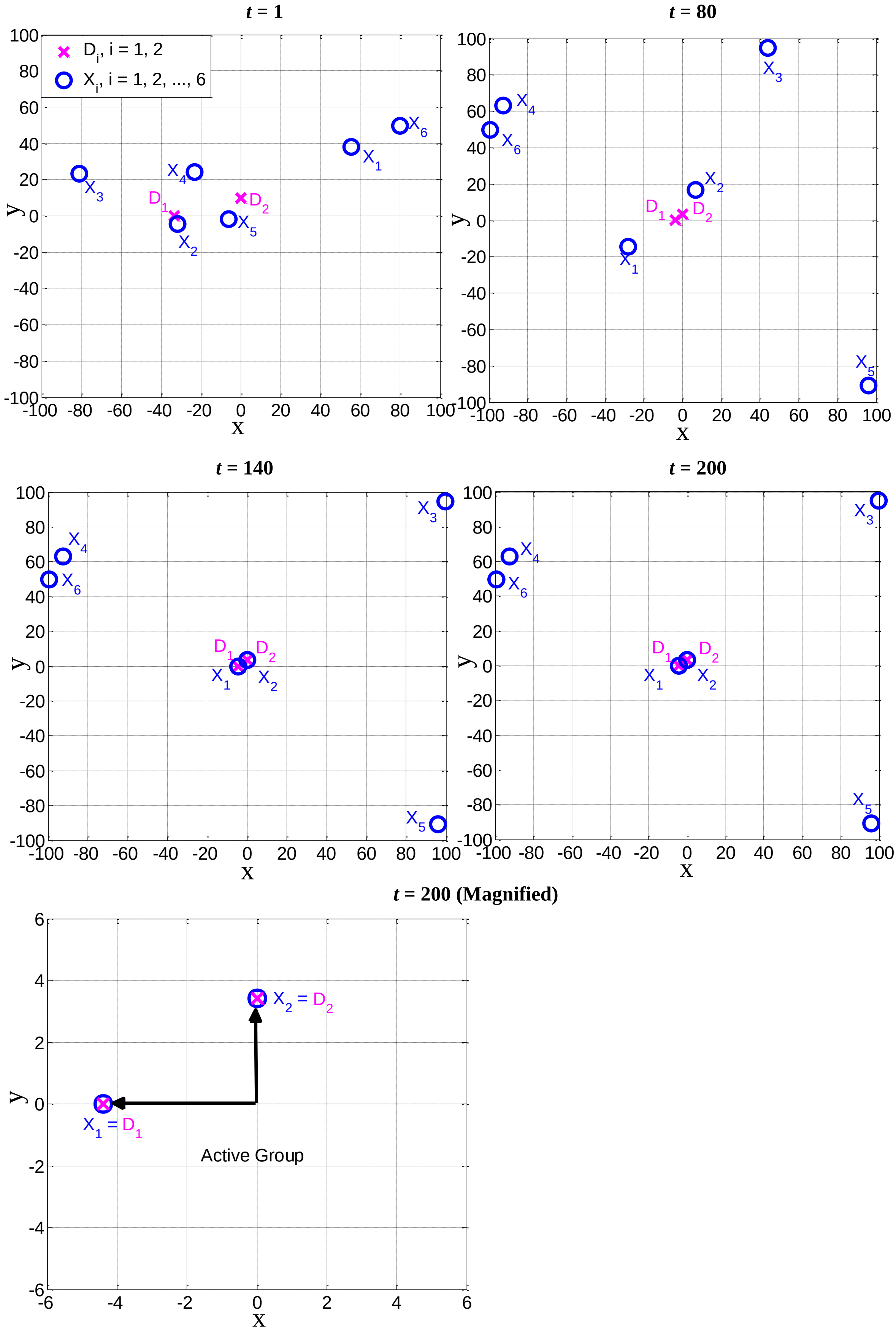


Fig. 8. Movement of positions X_i, i = 1, 2, ...,6 of six particles based on orthogonal vectors D_1 and D_2, two active group particles (1 and 2) and four passive group particles (3, 4, 5 and 6) searching for global solution [2.0,–3.0]

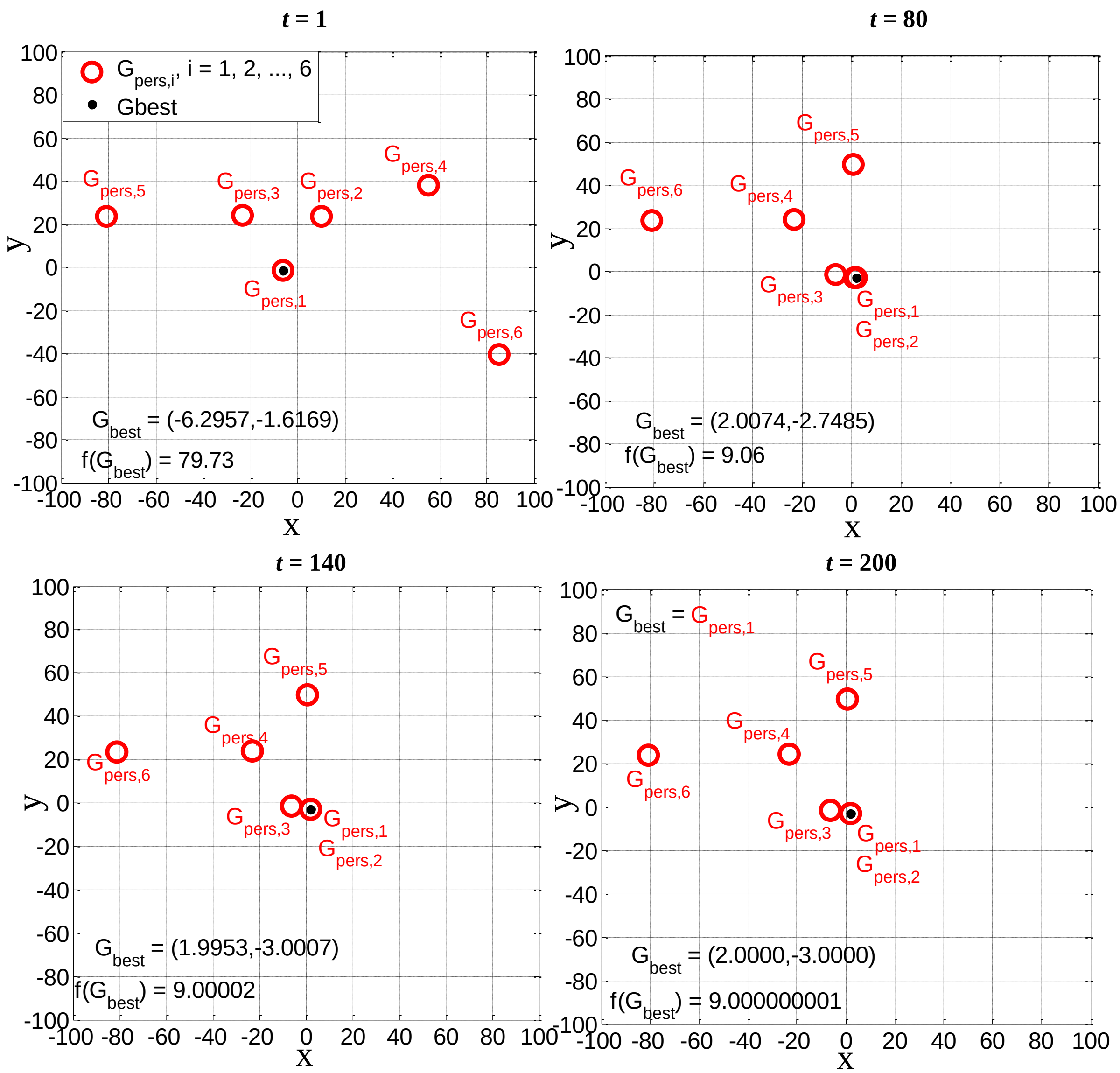


Fig. 9. Movement of the personal position vectors of six particles $G_{pers,i}$, i = 1, 2, ..., 6 and global best position vector G_{best}, two active group particles (1 and 2) and four passive group particles (3, 4, 5 and 6) searching for global solution [2.0,–3.0].

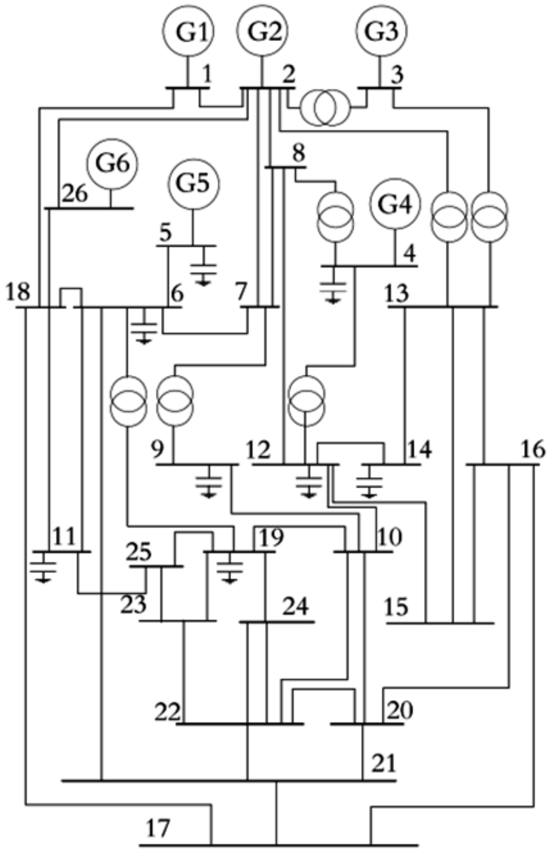


Fig. 10. Single line diagram of PS-1.

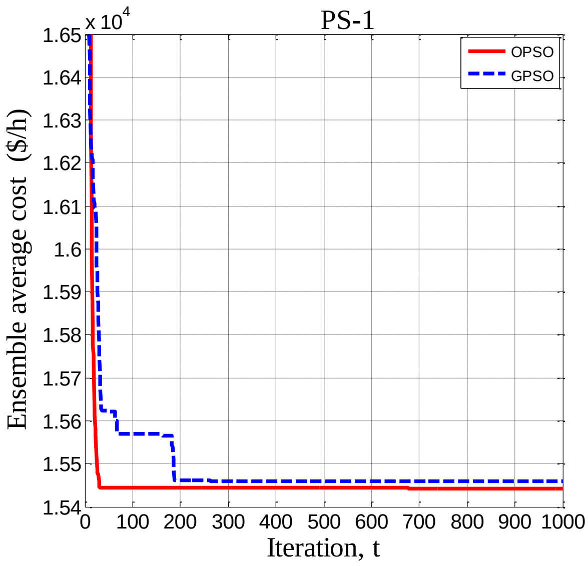


Fig. 11**. Convergence characteristics of OPSO and GPSO algorithms for PS-1.**

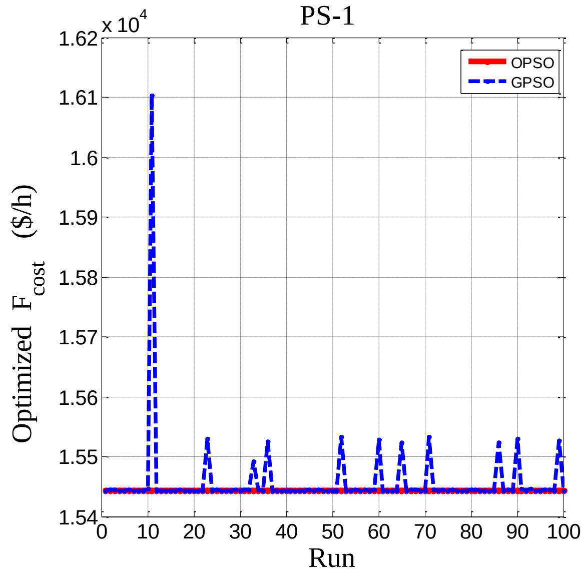


Fig. 12. Comparison of optimized cost per run between OPSO and GPSO algorithms for PS-1.

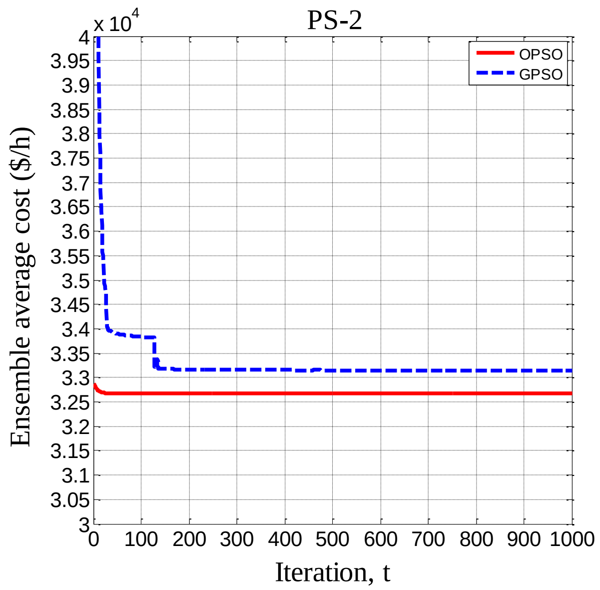


Fig. 13. Convergence characteristics of OPSO and GPSO algorithms for PS-2

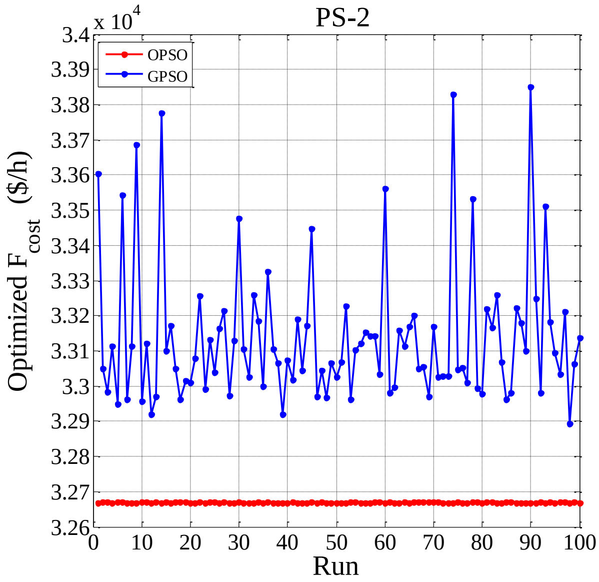


Fig. 14. Comparison of optimized cost per run between OPSO and GPSO algorithms for PS-2

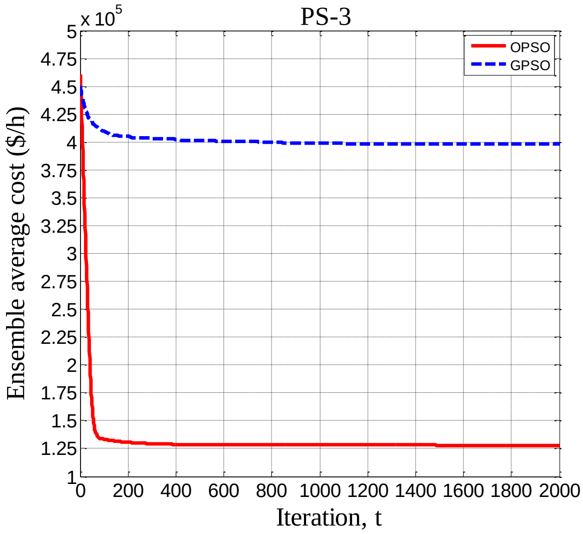


Fig. 15. Convergence characteristics of OPSO and GPSO algorithms for PS-3

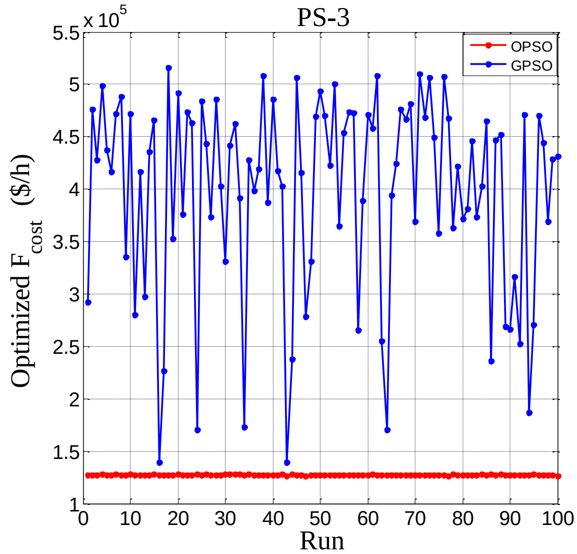
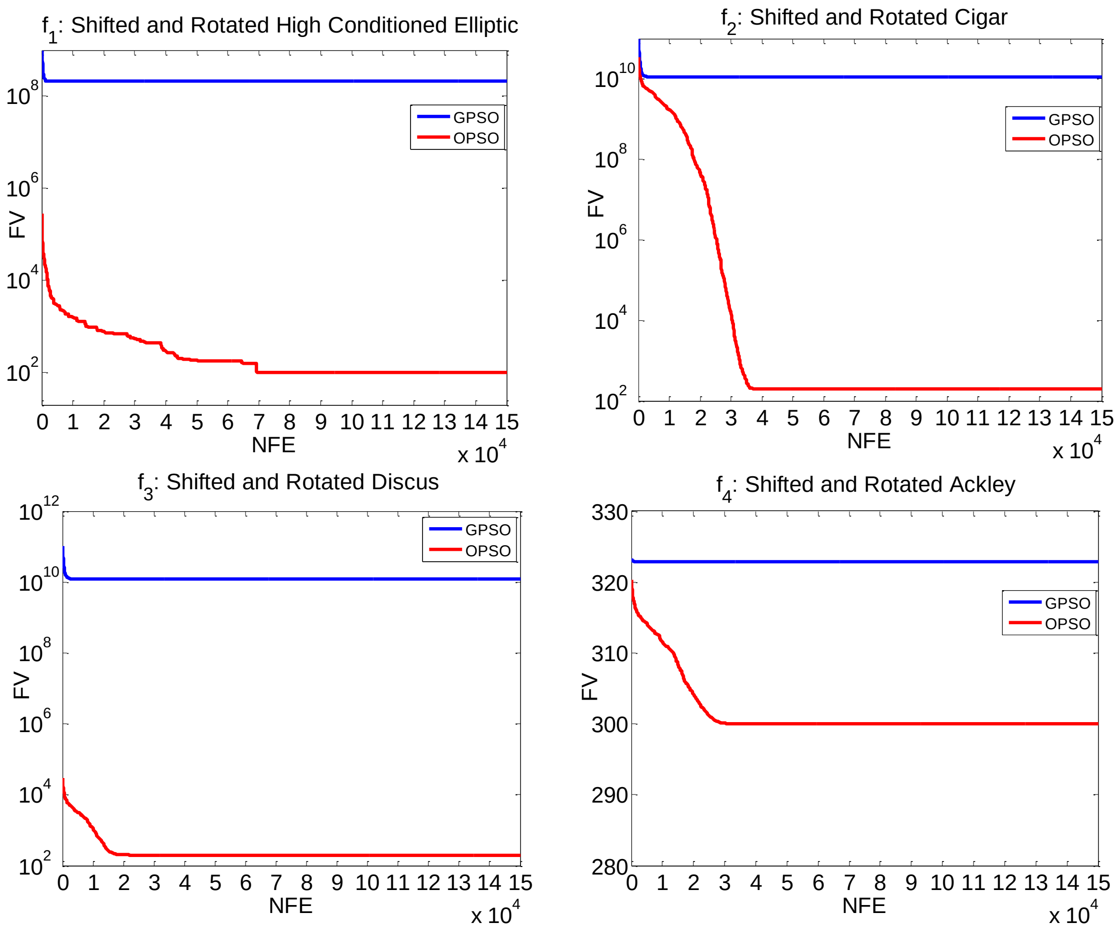
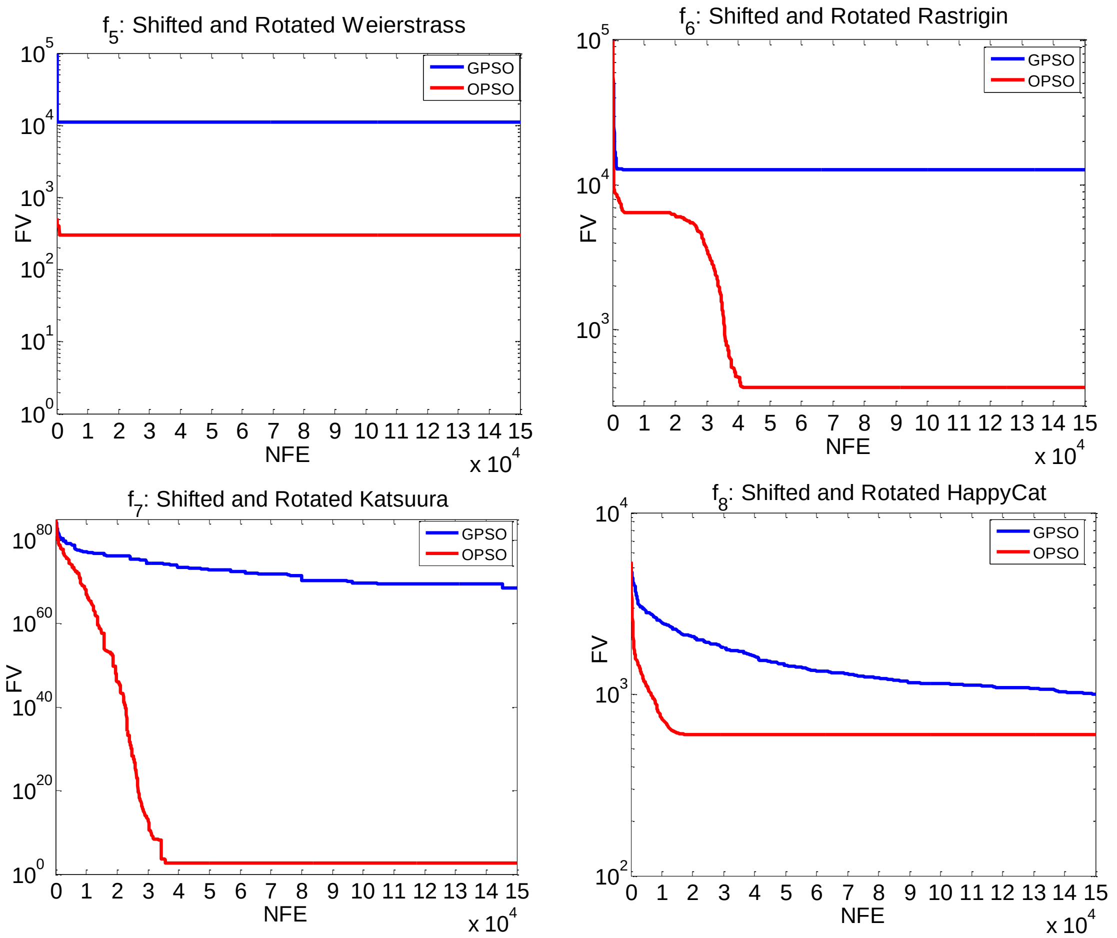


Fig. 16. Comparison of optimized cost per run between OPSO and GPSO algorithms for PS-3





Continued

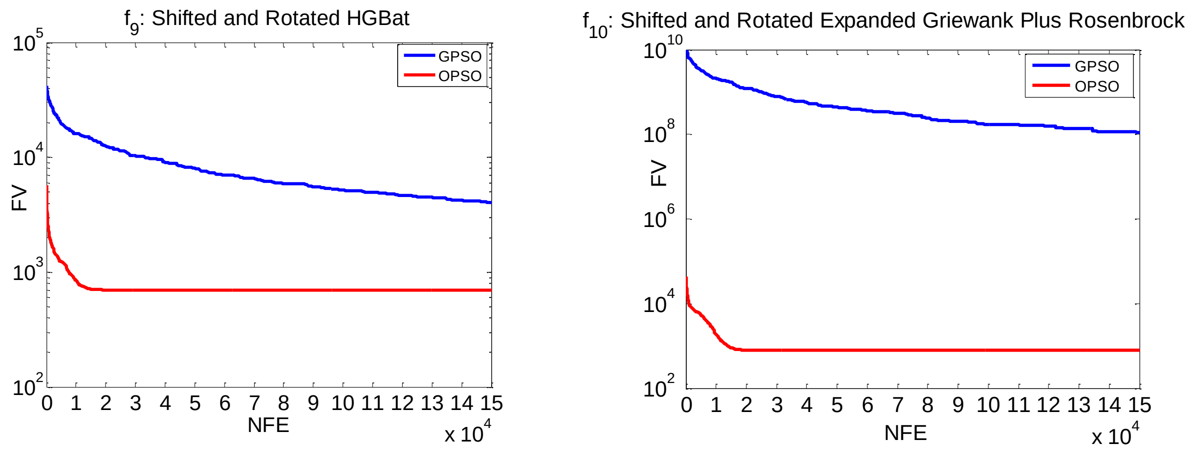


Fig. 17. Comparison of convergence characteristics between GPSO and OPSO algorithms for $f_1 - f_{10}$.

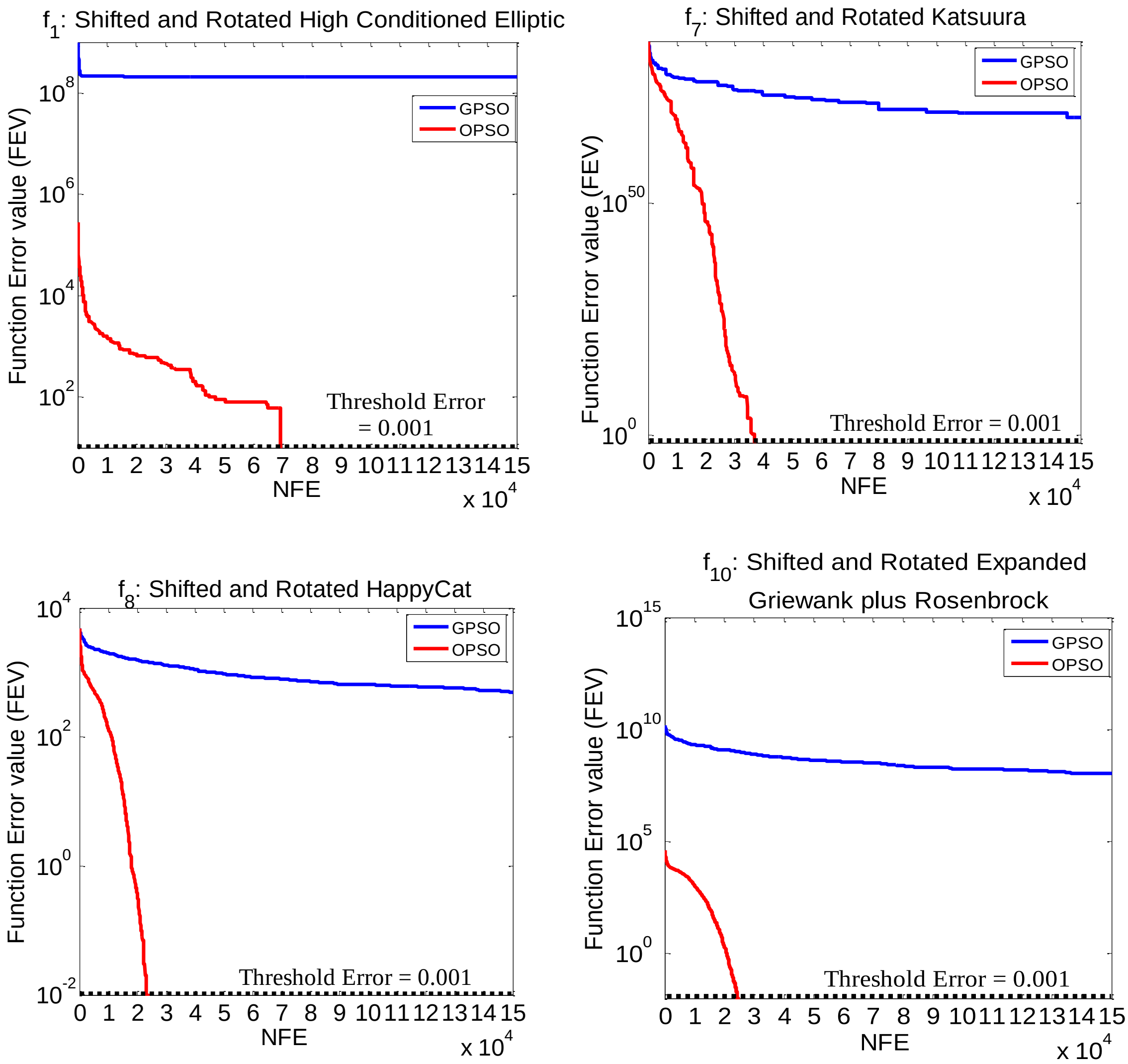


Fig. 18. The FEVs obtained at different runs at each NFE by the GPSO and OPSO algorithms for f_1, $f_7 - f_8$ and f_{10}.

Table 1: Specifications and power constraints for PS-1

TGU	P_j^0 (MW)	$P_{j,min}$ (MW)	$P_{j,max}$ (MW)	a_i ($)	b_i ($/MW)	c_i ($/MW^2)	UR_j (MW/h)	DR_j (MW/h)	POZs (MW)
1	440	100	500	240	7.0	0.0070	80	120	[210,240][350,380]
2	170	50	200	200	10.0	0.0095	50	90	[90,110][140,160]
3	200	80	300	220	8.5	0.0090	65	100	[150,170][210,240]
4	150	50	150	200	11.0	0.0090	50	90	[80,90][110,120]
5	190	50	220	220	10.5	0.0080	50	90	[90,110][140,150]
6	110	50	120	190	12.0	0.0075	50	90	[75,85][100,105]

Table 2: *B*-loss coefficients of 6 TGUs of PS-1

$B_{jk} = 1\times10^{-06}$ MW^{-1}

B_{jk}	*1*	*2*	*3*	*4*	*5*	*6*
1	17	12	7	-1	-5	-2
2	12	14	9	1	-6	-1
3	7	9	31	0	-10	-6
4	-1	1	0	24	-6	-8
5	-5	-6	-10	-6	129	-2
6	-2	-1	-6	-8	-2	150

Table 3: Comparison of cost performance between OPSO and other 22 ECTs for PS-1

Sl.No.	Algorithm	F_{min} ($/h)	F_{max} ($/h)	F_{mean} ($/h)	σ ($/h)	*R* ($/h)	AET (sec)
1	SOH-PSO [3]	15,446.0200	15,609.6400	15,497.3500	NA	136.6200	8.8620
2	GA [25]	15,445.5961	15,491.4797	15,465.1757	9.7336	45.8836	NA
3	DE [25]	15,444.9466	15,472.0651	15,450.1339	6.9854	27.1185	NA
4	ACSA [25]	15,445.3052	15,511.5269	15,459.5170	12.0247	66.2217	NA
5	AIS [25]	15,446.3283	15,481.2766	15,456.6660	7.3954	34.9483	NA
6	FA [25]	15,445.9448	15,501.3958	15,461.3003	9.3385	55.4510	NA
7	BCO [25]	15,444.5837	15,482.3963	15,457.9441	8.4816	37.8126	NA
8	APSO [25]	15,445.5109	15,538.6016	15,473.3164	12.9048	93.0907	NA
9	HGPSO [25]	15,447.1055	15,497.0335	15,462.6151	10.6456	49.9280	NA
10	HPSOM [25]	15,443.6281	15,479.8640	15,449.2603	6.2745	36.2359	NA
11	HPSOWM [25]	15,442.8205	15,502.6333	15,455.6220	15.8867	59.8128	NA
12	RDPSO [25]	15,442.7575	15,455.2936	15,445.0245	2.2828	12.5361	NA
13	SA-PSO [26]	15,447.0000	15,455.0000	15,447.0000	2.5280	8.0000	7.5800
14	NPSO-LRS [27]	15,450.0000	15,452.0000	15,450.5000	NA	2.0000	NA
15	CPSO [29]	15,446.0000	15,490.0000	15,449.0000	NA	44.0000	8.1300
16	QMPSO [30]	15,457.3380	15,489.9270	15,472.1840	4.5270	32.5890	7.1000
17	FDA [38]	15,449.5826	15,449.6508	15,449.6171	NA	0.0682	3.6340
18	GA-API [40]	15,449.7800	15,449.8100	15,449.8500	NA	**0.0300**	NA
19	MIQCQP [41]	15,443.0700	NA	NA	NA	NA	**0.4500**
20	λ-logic [43]	15,449.7960	NA	NA	NA	NA	NA
21	SPPO [46]	15,450. 0000	NA	NA	NA	NA	NA
22	GPSO	15,442.8334	16,103.3400	15,458.4000	69.6797	660.5074	2.5211
23	**OPSO**	15,442.8720	15,443.1996	**15,443.9754**	**0.2116**	0.3276	2.6334

Table 4: Optimized output power for each TGU obtained by OPSO and GPSO algorithms for PS-1

Algorithm	Optimum output power (MW)						Total output power (MW)
	P_1	P_2	P_3	P_4	P_5	P_6	
GPSO	450.5975	172.5329	261.3800	140.0706	164.4095	86.3979	1,275.3884
OPSO	450.0003	174.1009	261.9800	140.0674	162.3809	86.8386	1,275.3681

Table 5: Comparison of power balance constraint among 11 ECTs for PS-1

Sl.No.	Algorithm	Total P_j (MW)	P_D (MW)	P_{L1} (MW)	P_{L2} (MW)	$P_{L,\text{mismatch}}$ (MW)
1	SOH-PSO [3]	1,275.5500	1,263	12.5500	12.5500	**0.0000**
2	DRPSO [25]	1,275.3565	1,263	12.3565	12.3598	0.0033
3	NPSO-LRS [27]	1,275.9400	1,263	12.9400	12.9361	0.0039
4	CPSO [29]	1,276.0000	1,263	13.0000	12.9582	–0.0418
5	QMPSO [30]	1,275.3159	1,263	12.3159	12.4058	0.0899
6	GA-API [40]	1,276.1300	1,263	13.1300	13.1300	**0.0000**
7	MIQCQP [41]	1,275.4400	1,263	12.4400	12.4400	**0.0000**
8	λ-logic [43]	1,275.9500	1,263	12.9500	12.9500	**0.0000**
9	SPPO [46]	1,275.9700	1,263	12.9700	12.9700	**0.0000**
10	GPSO	1,275.3884	1,263	12.3884	12.3883	–0.0001
11	**OPSO**	1,275.3681	1,263	12.3681	12.3681	**0.0000**

Table 6: Specifications and power constraints for PS-2

TGU	P_j^0 (MW)	$P_{j,min}$ (MW)	$P_{j,max}$ (MW)	a_i ($)	b_i ($/MW)	c_i ($/MW$^{2)}$)	UR_j (MW/h)	DR_j (MW/h)	POZs (MW)
1	400	150	455	671	10.1	0.000299	80	120	-
2	300	150	455	574	10.2	0.000183	80	120	[185,225][305,335][450,450]
3	105	20	130	374	8.8	0.001126	130	130	-
4	100	20	130	374	8.8	0.001126	100	130	-
5	90	150	470	461	10.4	0.000205	80	120	[180,200][305,335][390,420]
6	400	135	460	630	10.1	0.000301	80	120	[230,225][365,395][430,455]
7	350	135	465	548	9.8	0.000364	80	120	-
8	95	60	300	227	11.2	0.000338	65	100	-
9	105	25	162	173	11.2	0.000807	60	100	-
10	110	25	160	175	10.7	0.001203	60	100	-
11	60	20	80	186	10.2	0.003586	80	80	-
12	40	20	80	230	9.9	0.005513	80	80	[30,40][55,65]
13	30	25	85	225	13.1	0.000371	80	80	-
14	30	15	55	309	12.1	0.001929	55	55	-
15	20	15	55	323	12.4	0.004447	55	55	-

Table 7: *B*-loss coefficients of 15 TGUs of PS-2

$B_{jk} = 10^{-06} \times MW^{-1}$

B_{jk}	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
1	14	12	7	-1	-3	-1	-1	-1	-3	5	-3	-2	4	3	-1
2	12	15	13	0	-5	-2	0	1	-2	-4	-4	-0	4	10	-2
3	7	13	76	-1	-13	-9	-1	0	-8	-12	-17	-0	-26	111	-28
4	-1	0	-1	34	-7	-4	11	50	29	32	-11	-0	1	1	-26
5	-3	5	-13	-7	90	14	-3	-12	-10	-13	7	-2	-2	-24	-3
6	-1	-2	-9	-4	14	16	-0	-6	-5	-8	11	-1	-2	-17	3
7	-1	0	-1	11	-3	-0	15	17	15	9	-5	7	-0	-2	-8
8	-1	1	0	50	-12	-6	17	168	82	79	-23	-36	1	5	-78
9	-3	-2	-8	29	-10	-5	15	82	129	116	-21	-25	7	-12	-72
10	-5	-4	-12	32	-13	-8	9	79	116	200	-27	-34	9	-11	-88
11	-3	-4	-17	-11	7	11	-5	-23	-21	-27	140	1	4	-38	168
12	-2	-0	-0	-0	-2	-1	7	-36	-25	-34	1	54	-1	-4	28
13	4	4	-26	1	-2	-2	-0	1	7	9	4	-1	103	-101	28
14	3	10	111	1	-24	-17	-2	5	-12	-11	-38	-4	-101	578	-94
15	-1	-2	-28	-26	-3	3	-8	-78	-72	-88	168	28	28	-94	1283

Table 8: Comparison of cost performance between OPSO and other 18 ECTs for PS-2

Sl.No.	Algorithm	Min. Cost ($/h)	Max. Cost ($/h)	Mean Cost ($/h)	σ ($/h)	*R* ($/h)	AET (sec)
1	PSO-MSAF [22]	32,713.0900	32,798.2500	32,759.6400	NA	85.1600	19.1500
2	GA [25]	32,939.5208	33,231.6216	33,106.0019	100.1279	292.1008	NA
3	DE [25]	32,818.5792	33,116.9340	32,990.8673	61.5145	298.3548	NA
4	ACSA [25]	32,785.6031	33,185.2761	33,051.7711	77.8005	399.6730	NA
5	AIS [25]	32,895.9173	33,132.0191	33,017.6537	58.1230	236.1018	NA
6	FA [25]	32,901.6610	33,197.2718	33,081.0107	91.0111	295.6108	NA
7	BCO [25]	32,989.2341	33,301.4940	33,113.0149	69.7986	312.2599	NA
8	APSO [25]	32,687.9840	33,359.6609	32,948.0533	92.0040	671.6769	NA
9	HGPSO [25]	32,864.0501	33,280.2655	33,034.1894	63.9932	416.2154	NA
10	HPSOM [25]	32,697.2458	33,015.7284	32,819.5931	83.0907	318.4826	NA
11	HPSOWM [25]	32,696.9585	33,034.3413	32,805.7185	87.8689	337.3828	NA
12	RDPSO [25]	32,666.1818	32,934.3089	32,739.7165	56.7070	268.1271	NA
13	SA-PSO [26]	32,708.0000	32,789.0000	32,732.0000	18.0250	81.0000	12.7900
14	CPSO [29]	32,834.0000	33,318.0000	33,021.0000	NA	484.0000	13.1300
15	EGSSOA [42]	NA	NA	32,680.1038	NA	NA	NA
16	λ-logic [43]	32,713.9510	NA	NA	NA	NA	NA
17	SPPO [46]	32,713.2100	NA	NA	NA	NA	NA
18	GPSO	32,891.8329	33,850.9528	33,137.5549	197.3331	959.1199	**3.5901**
19	**OPSO**	32,668.4863	32,669.3005	**32,668.9205**	**0.1394**	**0.8142**	4.3777

Table 9: Optimized output power for each TGU obtained by OPSO and GPSO algorithms for PS-2

Optimum output power (MW)		
TGU	GPSO	**OPSO**
1	455.0000	455.0000
2	377.1693	380.0000
3	125.8555	130.0000
4	113.9563	129.9696
5	161.2816	170.0000
6	458.6809	457.3942
7	418.2693	430.0000
8	152.0496	71.6613
9	78.3982	58.2340
10	71.8736	160.0000
11	60.5822	80.0000
12	65.7008	80.0000
13	34.3022	25.0000
14	50.5299	15.0000
15	36.3461	15.0000
Total output power (MW)	2,659. 9955	2,657.2591

Table 10: Comparison of power balance constraint among 9 ECTs for PS-2

Sl.No.	Algorithm	Total P_j MW	P_D MW	P_{L1} MW	P_{L2} MW	$P_{L,\mathrm{mismatch}}$ MW
1	PSO-MSAF [22]	2,660.4900	2,630	30.4900	30.4900	**0.0000**
2	DRPSO [25]	2,655.3650	2,630	25.3650	25.3696	0.0460
3	SA-PSO [26]	2,660.9000	2,630	30.9000	30.9080	0.0080
4	CPSO [29]	2,662.1000	2,630	32.1000	32.1303	0.0303
5	EGSSOA [42]	2,657.0120	2,630	27.0120	27.0120	**0.0000**
6	λ-logic [43]	2,659.9491	2,630	29.9491	29.9491	**0.0000**
7	SPPO [46]	2,660.0000	2,630	30.0000	31.4300	1.4300
8	GPSO	2,659.9955	2,630	29.9950	29.9971	0.0016
9	**OPSO**	2,657.2591	2,630	27.2591	27.2591	**0.0000**

Table 11: Specifications and power constraints for PS-3

TGU	P_j^0 (MW)	$P_{j,min}$ (MW)	$P_{j,max}$ (MW)	a_i (\$/h)	b_i (\$/MWh)	c_i (\$/MW^{2}h)	UR_j (MW/h)	DR_j (MW/h)	POZs (MW)
1	50	40	80	170.77	8.336	0.03073	35	60	-
2	60	60	120	309.54	7.0706	0.02028	40	70	[80,85]
3	150	80	190	369.03	8.1817	0.00942	50	90	[82,88]
4	24	24	42	135.48	6.9467	0.08482	42	42	-
5	42	26	42	135.19	6.5595	0.09693	42	42	-
6	75	68	140	222.33	8.0543	0.01142	40	75	-
7	100	110	300	287.71	8.0323	0.00357	65	100	[155,162][221,235]
8	152	135	300	391.98	6.9990	0.00492	65	100	-
9	200	135	300	455.76	6.6020	0.00573	65	100	[235,246]
10	100	130	300	722.82	12.908	0.00605	65	100	[200,211]
11	300	94	375	635.20	12.986	0.00515	55	95	[213,220]
12	300	94	375	654.69	12.796	0.00569	55	95	[213,220]
13	150	125	500	913.40	12.501	0.00421	80	120	[201,211][290,310][413,425]
14	200	125	500	1760.4	8.8412	0.00752	80	120	[205,217][306,318][409,420]
15	190	125	500	1728.3	9.1575	0.00708	80	120	[214,230][277,290][402,412]
16	190	125	500	1728.3	9.1575	0.00708	80	120	[214,230][277,290][402,412]
17	190	125	500	1728.3	9.1575	0.00708	80	120	[214,230][277,290][402,412]
18	400	220	500	647.85	7.9691	0.00313	70	110	[307,321][407,421]
19	400	220	500	649.69	7.9550	0.00313	70	110	[301,310][421,431]
20	398	242	500	647.83	7.9691	0.00313	70	110	[340,351][421,431]
21	398	242	500	647.81	7.9691	0.00313	70	110	[340,351][421,431]
22	390	254	550	785.96	6.6313	0.00298	70	110	[306,320][440,445]
23	390	254	550	785.96	6.6313	0.00298	70	110	[306,320][440,445]
24	390	254	550	794.53	6.6311	0.00284	70	110	[370,390][495,502]
25	390	254	550	794.53	6.6311	0.00284	70	110	[370,390][495,502]
26	390	254	550	801.32	7.1032	0.00277	70	110	[380,410][501,520]
27	390	254	550	801.32	7.1032	0.00277	70	110	[380,410][501,520]
28	20	10	150	1055.1	3.3353	0.52124	90	150	[102,113]
29	20	10	150	1055.1	3.3353	0.52124	90	150	[102,113]
30	30	10	150	1055.1	3.3353	0.52124	90	150	[102,113]
31	30	20	70	1207.8	13.052	0.25098	70	70	-
32	40	20	70	810.79	21.887	0.16766	70	70	-
33	40	20	70	1247.7	10.244	0.2635	70	70	-
34	25	20	70	1219.2	8.3707	0.30575	70	70	-
35	25	18	60	641.43	26.258	0.18362	60	60	-
36	20	18	60	1112.8	9.6956	0.32563	60	60	-
37	20	20	60	1044.4	7.1633	0.33722	60	60	-
38	25	25	60	832.24	16.339	0.23915	60	60	-
39	25	25	60	832.24	16.339	0.23915	60	60	-
40	25	25	60	1035.2	16.339	0.23915	60	60	-

Table 12: Comparison of cost performance between OPSO and other 14 ECTs for PS-3

Sl.No.	Algorithm	Min.Cost ($/h)	Max. Cost ($/h)	Mean Cost ($/h)	σ ($/h)	*R* ($/h)	AET (sec)
Without POZs, RRLs and P_L							
1	GA [25]	133,435.6906	136,274.9726	135,012.4985	729.3536	2,839.2820	NA
2	DE [25]	129,915.5635	137,042.9461	130,600.2269	1,335.4343	7,127.3826	NA
3	ACSA [25]	131,167.3417	134,923.6245	132,844.7110	741.0843	3,756.2828	NA
4	AIS [25]	130,133.9214	132,703.1884	131,482.2767	561.7950	2,569.2670	NA
5	FA [25]	130,948.8466	134,997.9243	133,511.4572	747.3692	4,049.0777	NA
6	BCO [25]	130,337.7290	132,999.8803	131,733.9439	589.8034	2,662.1513	NA
7	APSO [25]	130,861.5242	134,044.6303	132,587.8486	675.0344	3,183.1061	NA
8	HGPSO [25]	132,072.2495	135,528.3862	134,012.5706	684.4951	3,456.1367	NA
9	HPSOM [25]	129,177.4413	131,281.3077	130,234.1694	529.5827	2,103.8664	NA
10	HPSOWM [25]	129,717.3557	132,303.5999	130,858.6741	591.7691	2,586.2442	NA
11	RDPSO [25]	128,864.4525	131,129.0861	129,482.0970	568.9333	2.264.6336	NA
With POZs, RRLs and P_L							
12	MIQCQP [41]	128,395.2900	NA	NA	NA	NA	**13.34**
13	λ-logic [43]	129,777.5300	NA	NA	NA	NA	NA
14	GPSO	139,051.1893	515,712.5007	398,122.4784	95,189.4180	376,661.3114	47.32
15	**OPSO**	126,489.6228	127,916.1972	**127,349.8324**	**302.3502**	**1,426.5744**	69.32

Table 13: Optimized output power for each TGU obtained by OPSO and GPSO algorithms for PS-3

Optimum output power (MW)		
TGU	GPSO	**OPSO**
1	70.6053	79.9959
2	87.4700	100.0000
3	177.0581	190.0000
4	**3.5760**	41.0390
5	30.3749	36.6804
6	114.6114	115.0000
7	151.8385	153.4231
8	216.0995	217.0000
9	264.3870	265.0000
10	164.1162	160.5088
11	344.3448	354.1711
12	345.5677	348.6364
13	228.4384	219.8558
14	263.4364	280.0000
15	268.9301	270.0000
16	232.0047	270.0000
17	268.6993	270.0000
18	469.5224	470.0000
19	464.1353	470.0000
20	467.6499	468.0000
21	466.5318	468.0000
22	447.0084	460.0000
23	458.6545	460.0000
24	440.7477	460.0000
25	446.9992	460.0000
26	457.2052	460.0000
27	459.8870	460.0000
28	99.6434	35.1200
29	99.9167	35.9365
30	99.2875	45.3153
31	23.7599	56.3309
32	63.3148	36.1663
33	65.5211	41.2330
34	20.1840	47.6815

35	40.1371	40.0000
36	57.9855	55.0525
37	59.5737	53.4175
38	56.4260	40.0487
39	43.1472	40.0000
40	49.1706	54.4607
Total output power (MW)	8,587.9672	8,588.0734

Table 14: Comparison of power balance constraint between 2 ECTs for PS-3

Algorithm	Total P_j MW	P_D MW	P_{L1} MW	P_{L2} MW	$P_{L,\mathrm{mismatch}}$ MW
λ-logic [43]	8,637.3300	8,550	87.3300	87.4037	0.0737
OPSO	8,588.0734	8,550	38.0734	38.1121	**0.0387**

Table 15: Ten benchmark objective functions used in the study

f	Name	Function	Threshold Error	Optimum (x)	Minimum $f_i(x)$
f_1	Shifted and Rotated High Conditioned Elliptic [31]	$f_1(x)=\sum_{i=1}^{d}(10^6)^{(\frac{i-1}{d-1})} Z_{1,i}^2+F_1,$ $Z_1 = M_1 \times (x - O_{1i})$, $x = [x_1, x_2, ..., x_d]$, $O_{1i} = [o_{11}, o_{12}, ..., o_{1d}]$	0.001	O_{1i}	$F_1=100$
f_2	Shifted and Rotated Cigar [31]	$f_2(x)=Z_{2,1}^2+10^6\sum_{i=2}^{d}Z_{2,i}^2+F_2,$ $Z_2 = M_2 \times (x - O_{2i})$, $x = [x_1, x_2, ..., x_d]$, $O_{2i} = [o_{21}, o_{22}, ..., o_{2d}]$	0.001	O_{2i}	$F_2=200$
f_3	Shifted and Rotated Discus [32]	$f_3(x)=10^6Z_{3,1}^2+\sum_{i=2}^{d}Z_{3,i}^2+F_3,$ $Z_3 = M_3 \times (x - O_{3i})$, $x = [x_1, x_2, ..., x_d]$, $O_{3i} = [o_{31}, o_{32}, ..., o_{3d}]$	0.001	O_{3i}	$F_3=200$
f_4	Shifted and Rotated Ackley [31]	$f_4(x)=-a\ exp\ (-b\sqrt{\frac{1}{d}\sum_{i=1}^{d}Z_{4,i}^2}\ )-exp\ (\frac{1}{d}\sum_{i=1}^{d}cos(cZ_{4,i}))+$ $a+exp\ (1)+F_4, a=20, b=0.2, c=2\pi$ $Z_4 = M_4 \times (x - O_{4i})$, $x = [x_1, x_2, ..., x_d]$, $O_{4i} = [o_{41}, o_{42}, ..., o_{4d}]$	0.001	O_{4i}	$F_4=300$
f_5	Shifted and Rotated Weierstrass [32]	$f_5(x)=\sum_{i=1}^{d}\left[\sum_{k=1}^{k_{max}}a^k\cos(2\pi b^k(Z_{5,i}+0.5))-d\sum_{k=1}^{k_{max}}a^k\cos(\pi b^k)\right]+F_5$ $a = 0.5$, $b = 3$, $k_{max} = 20$, $Z_5=M_5\times\left(\frac{0.5\times(x-O_{5i})}{100}\right)$, $x = [x_1, x_2, ..., x_d]$ $O_{5i} = [o_{51}, o_{52}, ..., o_{5d}]$	0.001	O_{5i}	$F_5=300$
f_6	Shifted and Rotated Rastrigin [31]	$f_6(x)=10\,d+\sum_{i=1}^{d}[Z_{6,i}^2-10\cos(2\pi Z_{6,i})]+F_6,$ $Z_6=M_6\times\left(\frac{5.12\times(x-O_{6i})}{100}\right)$, $x = [x_1, x_2, ..., x_d]$, $O_{6i} = [o_{61}, o_{62}, ..., o_{6d}]$	0.001	O_{6i}	$F_6=400$
f_7	Shifted and Rotated Katsuura [32]	$f_7(x)=\frac{10}{d^2}\prod_{i=1}^{d}\left[1+i\sum_{j=1}^{32}\frac{\lvert 2^i\times Z_{7,i}-round(2^i\times Z_{7,i})\rvert}{2^i}\right]^{\frac{10}{d^{1.2}}}-\frac{10}{d^2}+F_7$ $Z_7=M_7\times\left(\frac{5\times(x-O_{7i})}{100}\right)$, $x = [x_1, x_2, ..., x_d]$, $O_{7i} = [o_{71}, o_{72}, ..., o_{7d}]$	0.001	O_{7i}	$F_7=500$
f_8	Shifted and Rotated HappyCat [32]	$f_8(x)=\left[\sum_{i=1}^{d}Z_{8,i}^2-d\right]^{\frac{1}{4}}+(\frac{0.5\sum_{i=1}^{d}Z_{8,i}^2+\sum_{i=1}^{d}Z_{8,i})}{\mathrm{d}})+0.5+F_8,$ $Z_8=M_8\times\left(\frac{5\times(x-O_{8i})}{100}\right)$, $x = [x_1, x_2, ..., x_d]$, $O_{8i} = [o_{81}, o_{82}, ..., o_{8d}]$	0.001	O_{8i}	$F_8=600$
f_9	Shifted and Rotated HGBat [32]	$f_9(x)=\left[(\sum_{i=1}^{d}Z_{9,i}^2)^2-(\sum_{i=1}^{d}Z_{9,i})^2\right]^{\frac{1}{2}}+(\frac{0.5\sum_{i=1}^{d}Z_{9,i}^2+\sum_{i=1}^{d}Z_{9,i})}{\mathrm{d}})+0.5+F_9,$ $Z_9=M_9\times\left(\frac{5\times(x-O_{9i})}{100}\right)$, $x = [x_1, x_2, ..., x_d]$, $O_{9i} = [o_{91}, o_{92}, ..., o_{9d}]$	0.001	O_{9i}	$F_9=700$
f_{10}	Shifted and Rotated Expanded Griewank plus Rosenbrock [32]	$f_g(x)=\frac{1}{4000}\sum_{i=1}^{d}Z_{10,i}^2-\prod_{i=1}^{d}\cos(\frac{Z_{10,i}}{\sqrt{i}})+1$ $f_r(x)=\sum_{i=1}^{d-1}[100\ (Z_{10,i+1}-Z_{10,i}^2)^2+(Z_{10,i}-1)^2]$ $f_{10}(x)=f_g(f_r(Z_{10,1},Z_{10,2})+f_g(f_r(Z_{10,2},Z_{10,3})+\cdots+$ $f_g(f_r(Z_{10,d-1},Z_{10,d})+f_g(f_r(Z_{10,d},Z_{10,1})+F_{10}$ $Z_{10}=M_{10}\times\left(\frac{5\times(x-O_{10i})}{100}\right)+1$, $x = [x_1, x_2, ..., x_d]$, $O_{10i} = [o_{101}, o_{102}, ..., o_{10d}]$	0.001	O_{10i}	$F_{10}=800$

Table 16: Performance comparison between GPSO and OPSO algorithm on ten benchmark functions

f	Minimum $f(x)$	Threshold Error	Fitness	GPSO	OPSO	f	Minimum $f(x)$	Threshold Error	Fitness	GPSO	OPSO
f_1	100	1×10^{-03}	BFV	6.7217×10^{07}	1.0000×10^{02}	f_6	400	1×10^{-03}	BFV	6.3726×10^{03}	4.0000×10^{02}
			WFV	3.5895×10^{08}	1.0000×10^{02}				WFV	1.9427×10^{04}	4.0000×10^{02}
			MFV	2.1017×10^{08}	1.0000×10^{02}				MFV	1.2650×10^{04}	4.0000×10^{02}
			MFEV	2.1007×10^{08}	0.0000				MFEV	1.264×10^{04}	0.0000
			σ	8.1122×10^{07}	5.9432×10^{-50}				σ	3.3683×10^{03}	3.7342×10^{-44}
			AET (sec)	-	172.4597				AET (sec)	-	210.0116
f_2	200	1×10^{-03}	BFV	6.0323×10^{09}	2.0000×10^{02}	f_7	500	1×10^{-03}	BFV	1.7048×10^{10}	5.0000×10^{02}
			WFV	1.5370×10^{10}	2.0000×10^{02}				WFV	2.8180×10^{69}	5.0000×10^{02}
			MFV	1.1061×10^{10}	2.0000×10^{02}				MFV	3.8636×10^{68}	5.0000×10^{02}
			MFEV	1.1051×10^{10}	0.0000				MFEV	3.8626×10^{68}	0.0000
			σ	2.7141×10^{09}	3.3248×10^{-54}				σ	8.8480×10^{68}	1.8626×10^{-43}
			AET (sec)	-	73.0369				AET (sec)	-	114.8182
f_3	200	1×10^{-03}	BFV	5.2660×10^{09}	2.0000×10^{02}	f_8	600	1×10^{-03}	BFV	8.6363×10^{02}	6.0000×10^{02}
			WFV	1.8594×10^{10}	2.0000×10^{02}				WFV	1.1724×10^{03}	6.0000×10^{02}
			MFV	1.2112×10^{10}	2.0000×10^{02}				MFV	9.9938×10^{02}	6.0000×10^{02}
			MFEV	1.2102×10^{10}	0.0000				MFEV	9.9928×10^{02}	0.0000
			σ	3.8578×10^{09}	3.4503×10^{-54}				σ	7.4942×10^{01}	1.5208×10^{-43}
			AET (sec)	-	35.8556				AET (sec)	-	121.8450
f_4	300	1×10^{-03}	BFV	3.2275×10^{02}	3.0000×10^{02}	f_9	700	1×10^{-03}	BFV	2.6696×10^{03}	7.0000×10^{02}
			WFV	3.2283×10^{02}	3.0000×10^{02}				WFV	6.4626×10^{03}	7.0000×10^{02}
			MFV	3.2289×10^{02}	3.0000×10^{02}				MFV	4.0527×10^{03}	7.0000×10^{02}
			MFEV	3.2279×10^{02}	0.0000				MFEV	4.0517×10^{03}	0.0000
			σ	3.3191×10^{-02}	6.1167×10^{-44}				σ	9.2835×10^{02}	3.0416×10^{-43}
			AET (sec)	-	37.4985				AET (sec)	-	122.2512
f_5	300	1×10^{-03}	BFV	1.1001×10^{02}	3.0000×10^{02}	f_{10}	800	1×10^{-03}	BFV	3.1925×10^{07}	8.0000×10^{02}
			WFV	1.1002×10^{02}	3.0000×10^{02}				WFV	2.6340×10^{08}	8.0000×10^{02}
			MFV	1.1001×10^{02}	3.0000×10^{02}				MFV	1.1174×10^{08}	8.0000×10^{02}
			MFEV	1.0991×10^{02}	0.0000				MFEV	1.1164×10^{08}	0.0000
			σ	4.7370×10^{-02}	1.0935×10^{-41}				σ	6.9107×10^{07}	2.2737×10^{-41}
			AET (sec)	-	230.8762				AET (sec)	-	139.2776

Table 17: Sensitivity analysis for the OPSO algorithm with increasing swarm's size with d =30

f	Minimum $f(x)$	Fitness	m = 33	AET (sec)	m = 40	AET (sec)	m = 50	AET (sec)	m = 80	AET (sec)	m = 100	AET (sec)
f_1	100	BFV	100.00	127.44	100.00	188. 67	100.00	313.28	100.00	390.09	100.00	430.70
		WFV	100.00		100.00		100.00		100.00		100.00	
		MFV	100.00		100.00		100.00		100.00		100.00	
f_7	500	BFV	500.00	55.26	500.00	124.57	500.00	182.50	500.00	232.89	500.00	282.63
		WFV	500.00		500.00		500.00		500.00		500.00	
		MFV	500.00		500.00		500.00		500.00		500.00	
f_8	600	BFV	600.00	65.22	600.00	131.45	600.00	183.07	600.00	246.92	600.00	317.89
		WFV	600.00		600.00		600.00		600.00		600.00	
		MFV	600.00		600.00		600.00		600.00		600.00	
f_{10}	800	BFV	800.00	74.13	800.00	106.63	800.00	191.14	800.00	234.64	800.00	294.27
		WFV	800.00		800.00		800.00		800.00		800.00	
		MFV	800.00		800.00		800.00		800.00		800.00	

Table 18: Performance comparison between OPSO algorithm and four ECTs using four CEC 2015-LBP [31] benchmark functions with *d* = 30

ECTs	Performance measure	f_1	f_2	f_4	f_6
OPSO (proposed)	MFEV	**0.00**	**0.00**	**0.00**	**0.00**
	σ	**5.94×10^{-50}**	**3.32×10^{-54}**	**6.11×10^{-44}**	**3.73×10^{-44}**
LLUDE [33]	MFEV	5.93×10^{-01}	2.84×10^{-14}	2.03×10^{01}	2.59×10^{01}
	σ	2.47×10^{-01}	2.69×10^{-14}	2.33×10^{-02}	3.28×10^{00}
SaDE [33]	MFEV	1.78×10^{03}	2.38×10^{-11}	2.05×10^{01}	3.46×10^{01}
	σ	1.43×10^{03}	7.22×10^{-11}	5.99×10^{-02}	6.44×10^{00}
JADE [33]	MFEV	6.23×10^{00}	3.41×10^{-14}	2.03×10^{01}	2.61×10^{01}
	σ	1.55×10^{01}	1.17×10^{-14}	2.86×10^{-02}	3.39×10^{00}
CoDE [33]	MFEV	1.58×10^{04}	6.02×10^{-13}	2.00×10^{01}	2.97×10^{01}
	σ	1.16×10^{04}	9.88×10^{-13}	9.98×10^{-02}	1.08×10^{01}
SPS-L-SHADE-EIG [48] Rank #1 CEC 2015-LBP [31]	MFEV	**0.00**	**0.00**	2.00×10^{01}	1.03×10^{01}
	σ	**0.00**	**0.00**	7.29×10^{-05}	1.41×10^{01}
DEsPA [49] Rank #2 CEC 2015-LBP [31]	MFEV	**0.00**	**0.00**	2.01×10^{01}	9.71×10^{00}
	σ	**0.00**	**0.00**	4.36×10^{-02}	3.02×10^{00}
MVMO [50] Rank #3 CEC 2015-LBP [31]	MFEV	**0.00**	**0.00**	2.00×10^{01}	9.54×10^{00}
	σ	**0.00**	**0.00**	5.42×10^{-04}	3.53×10^{00}
Max.NFE		3.00×10^{05}	3.00×10^{05}	3.00×10^{05}	3.00×10^{05}

Table 19: Performance comparison between OPSO algorithm and four ECTs using six CEC 2015-EOP [32] benchmark functions with *d* = 30

ECTs	Performance measure	f_3	f_5	f_7	f_8	f_9	f_{10}
OPSO (proposed)	MFEV	6.81×10^{-02}	$\mathbf{1.37\times10^{00}}$	$\mathbf{6.40\times10^{-01}}$	$\mathbf{1.49\times10^{-02}}$	$\mathbf{1.54\times10^{-02}}$	$\mathbf{1.29\times10^{00}}$
	σ	2.96×10^{-02}	$\mathbf{3.35\times10^{-01}}$	$\mathbf{4.00\times10^{-01}}$	$\mathbf{5.97\times10^{-03}}$	$\mathbf{7.45\times10^{-03}}$	6.74×10^{-02}
	AET (sec)	$\mathbf{2.32\times10^{00}}$	1.98×10^{01}	$\mathbf{4.51\times10^{00}}$	$\mathbf{1.47\times10^{00}}$	$\mathbf{1.43\times10^{00}}$	$\mathbf{1.60\times10^{00}}$
SbaDE **[34]**	MFEV	2.54×10^{09}	2.00×10^{01}	4.33×10^{02}	4.56×10^{04}	7.52×10^{01}	2.39×10^{07}
	σ	5.00×10^{09}	2.04×10^{02}	9.46×10^{01}	4.07×10^{04}	4.15×10^{03}	5.43×10^{07}
	AET (sec)	1.76×10^{01}	$\mathbf{1.65\times10^{01}}$	1.68×10^{01}	1.71×10^{01}	1.78×10^{01}	1.72×10^{01}
DD-SRPSO [35]	MFEV	2.59×10^{04}	2.24×10^{01}	2.71×10^{00}	5.61×10^{-01}	5.43×10^{-01}	3.32×10^{02}
	σ	1.05×10^{04}	2.12×10^{00}	7.58×10^{-01}	1.00×10^{-01}	2.03×10^{-01}	2.12×10^{02}
	AET (sec)	-	-	-	-	-	-
EPSO [36]	MFEV	6.37×10^{04}	3.38×10^{02}	5.04×10^{02}	6.02×10^{02}	7.21×10^{02}	1.27×10^{05}
	σ	-	-	-	-	-	-
	AET (sec)	-	-	-	-	-	-
SHPSO-GSA [37]	MFEV	6.96×10^{-01}	1.80×10^{01}	1.06×10^{00}	5.26×10^{-01}	1.36×10^{-01}	2.78×10^{00}
	σ	2.11×10^{01}	9.03×10^{-01}	5.55×10^{-01}	4.62×10^{-01}	3.11×10^{-01}	$\mathbf{1.04\times10^{-04}}$
	AET (sec)	-	-	-	-	-	-
MVMO [51] Rank #1 CEC 2015-EOP [32]	MFEV	$\mathbf{6.93\times10^{-03}}$	3.79×10^{01}	1.67×10^{01}	5.20×10^{-01}	4.39×10^{-01}	4.03×10^{02}
	σ	$\mathbf{3.24\times10^{-04}}$	3.85×10^{0}	5.04×10^{-01}	1.32×10^{-01}	9.93×10^{-02}	2.63×10^{02}
	AET (sec)	-	-	-	-	-	-
TunedCMAES [52] Rank #2 CEC 2015-EOP [32]	MFEV	1.17×10^{05}	3.21×10^{02}	5.05×10^{02}	6.00×10^{02}	7.00×10^{02}	8.22×10^{02}
	σ	2.19×10^{04}	5.06×10^{0}	5.91×10^{-01}	2.35×10^{-01}	2.86×10^{-01}	1.09×10^{-01}
	AET (sec)	-	-	-	-	-	-
Max.NFE		1.50×10^{03}	1.50×10^{03}	1.50×10^{03}	1.50×10^{03}	1.50×10^{03}	1.50×10^{03}

Table 20 : Statistical results of unpaired t-Test of OPSO algorithm against seven ECTs for CEC 2015-LBP [31]

Sl. No.	f	Statistical Results	Competitive Algorithms						
			LLUDE [33]	SaDE [33]	JADE [33]	CoDE [33]	SPS-L-SHADE-EIG [48] Rank #1 CEC 2015-LBP [31]	DEsPA [49] Rank #2 CEC 2015-LBP [31]	MVMO [50] Rank #3 CEC 2015-LPB [31]
1	f_1	t-value	$-\infty$	$-\infty$	$\mathbf{-1.53\times10^{16}}$	$-\infty$	0.0	0.0	0.0
		p-value	**0**	**0**	$\mathbf{7.99\times10^{-297}}$	**0**	5.00×10^{-01}	5.00×10^{-01}	5.00×10^{-01}
2	f_2	t-value	0.0	0.0	0.0	0.0	0.0	0.0	0.0
		p-value	5.00×10^{-01}	5.00×10^{-01}	5.00×10^{-01}	5.00×10^{-01}	5.00×10^{-01}	5.00×10^{-01}	5.00×10^{-01}
3	f_4	t-value	$\mathbf{-2.49\times10^{16}}$	$\mathbf{-2.00\times10^{16}}$	$\mathbf{-2.49\times10^{16}}$	$\mathbf{-2.47\times10^{16}}$	$\mathbf{-2.47\times10^{16}}$	$\mathbf{-2.47\times10^{16}}$	$\mathbf{-2.47\times10^{16}}$
		p-value	$\mathbf{7.50\times10^{-301}}$	$\mathbf{4.74\times10^{-299}}$	$\mathbf{7.50\times10^{-301}}$	$\mathbf{9.05\times10^{-301}}$	$\mathbf{9.05\times10^{-301}}$	$\mathbf{9.05\times10^{-301}}$	$\mathbf{9.05\times10^{-301}}$
4	f_6	t-value	$\mathbf{-1.06\times10^{16}}$	$\mathbf{-2.12\times10^{16}}$	$\mathbf{-1.08\times10^{16}}$	$-\infty$	$\mathbf{-2.53\times10^{16}}$	$\mathbf{-1.19\times10^{16}}$	$\mathbf{-1.17\times10^{16}}$
		p-value	$\mathbf{8.51\times10^{-294}}$	$\mathbf{1.56\times10^{-299}}$	$\mathbf{7.35\times10^{-294}}$	**0**	$\mathbf{5.67\times10^{-301}}$	$\mathbf{9.13\times10^{-295}}$	$\mathbf{1.28\times10^{-294}}$
t = negative $t < 0$			3	3	3	3	2	2	2
t = positive $t = \geq 0$			1	1	1	1	2	2	2

General Merit Over Contender	**2**	**2**	**2**	**2**	0	0	0

Table 21: Statistical results of unpaired t-Test OPSO algorithm against six ECTs for CEC 2015-EOP [32]

Sl. No.	f	Statistical Results	Competitive Algorithms					
			SbaDE [34]	DD-SRPSO [35]	EPSO [36]	SHPSO-GSA [37]	MVMO [51] Rank #1 CEC 2015-EOP [32]	Tuned CMAES [52] Rank #2 CEC 2015-EOP [32]
1	f_3	t-value	**-3.80×10^{11}**	**-3.88×10^{06}**	**-9.54×10^{06}**	**-9.41×10^{01}**	1.33×10^{00}	**-1.75×10^{07}**
		p-value	**2.37×10^{-209}**	**1.63×10^{-114}**	**6.14×10^{-122}**	**7.89×10^{-27}**	9.80×10^{-02}	**5.90×10^{-127}**
2	f_5	t-value	**-2.49×10^{02}**	**-2.81×10^{02}**	**-4.49×10^{03}**	**-2.22×10^{02}**	**-4.87×10^{02}**	**-4.27×10^{03}**
		p-value	**7.69×10^{-35}**	**7.69×10^{-36}**	**1.00×10^{-58}**	**6.66×10^{-34}**	**2.13×10^{-40}**	**2.69×10^{-58}**
3	f_7	t-value	**-6.49×10^{04}**	**-3.96×10^{02}**	**-7.55×10^{04}**	**-1.48×10^{02}**	**-2.49×10^{03}**	**-7.56×10^{04}**
		p-value	**9.44×10^{-81}**	**1.12×10^{-38}**	**5.27×10^{-82}**	**1.35×10^{-30}**	**7.39×10^{-54}**	**5.07×10^{-82}**
4	f_8	t-value	**-3.40×10^{07}**	**-4.09×10^{02}**	**-4.50×10^{05}**	**-3.82×10^{02}**	**-3.78×10^{02}**	**-4.48×10^{05}**
		p-value	**1.93×10^{-132}**	**6.07×10^{-39}**	**9.85×10^{-97}**	**2.13×10^{-38}**	**2.67×10^{-38}**	**1.05×10^{-96}**
5	f_9	t-value	**-4.07×10^{04}**	**-2.85×10^{02}**	**-3.90×10^{05}**	**-1.01×10^{01}**	**-2.29×10^{02}**	**-3.79×10^{05}**
		p-value	**6.56×10^{-77}**	**5.50×10^{-36}**	**1.45×10^{-95}**	**8.08×10^{-24}**	**3.56×10^{-34}**	**2.55×10^{-95}**
6	f_{10}	t-value	**-1.59×10^{09}**	**-2.21×10^{04}**	**-8.48×10^{06}**	**-1.77×10^{02}**	**-2.69×10^{04}**	**-5.49×10^{04}**
		p-value	**3.46×10^{-164}**	**6.77×10^{-72}**	**5.71×10^{-121}**	**4.79×10^{-32}**	**1.70×10^{-73}**	**2.22×10^{-79}**
t = negative $t < 0$			6	6	6	6	5	6
t = positive $t = \geq 0$			0	0	0	0	1	0
General Merit Over Contender			**6**	**6**	**6**	**6**	**4**	**6**

Table 22: Statistical results of unpaired t-Test of OPSO algorithm against twenty four ECTs for PS-1, PS-2 and PS-3

Sl. No.	Competitive Algorithms	PS-1		PS-2		PS-3		t = negative	t = positive	General Merit Over
		t-value	p-value	t-value	p-value	t-value	p-value	$t < 0$	$t \geq 0$	Contender
1	GA [25]	**-1.03×10^{03}**	**1.14×10^{-201}**	**-3.14×10^{04}**	**0.0**	**-2.53×10^{02}**	**4.69×10^{-141}**	3	0	**3**
2	DE [25]	**-3.27×10^{02}**	**4.33×10^{-152}**	**-2.31×10^{04}**	**0.0**	**-1.07×10^{02}**	**2.48×10^{-104}**	3	0	**3**
3	ACSA [25]	**-7.71×10^{02}**	**7.23×10^{-189}**	**-2.75×10^{04}**	**0.0**	**-1.82×10^{02}**	**8.67×10^{-127}**	3	0	**3**
4	AIS [25]	**-6.36×10^{02}**	**1.35×10^{-180}**	**-2.50×10^{04}**	**0.0**	**-1.48×10^{02}**	**3.68×10^{-118}**	3	0	**3**
5	FA [25]	**-8.55×10^{02}**	**2.51×10^{-193}**	**-2.96×10^{04}**	**0.0**	**-2.04×10^{02}**	**1.06×10^{-131}**	3	0	**3**
6	BCO [25]	**-6.96×10^{02}**	**1.64×10^{-184}**	**-3.19×10^{04}**	**0.0**	**-1.45×10^{02}**	**4.08×10^{-117}**	3	0	**3**
7	APSO [25]	**-1.42×10^{03}**	**3.25×10^{-215}**	**-2.00×10^{04}**	**0.0**	**-1.73×10^{02}**	**9.77×10^{-125}**	3	0	**3**
8	HGPSO [25]	**-9.17×10^{02}**	**2.43×10^{-196}**	**-2.62×10^{04}**	**0.0**	**-2.20×10^{02}**	**4.70×10^{-135}**	3	0	**3**
9	HPSOM [25]	**-2.86×10^{02}**	**2.63×10^{-146}**	**-1.08×10^{04}**	**1.64×10^{-302}**	**-9.54×10^{01}**	**3.04×10^{-99}**	3	0	**3**
10	HPSOWM [25]	**-5.86×10^{02}**	**3.88×10^{-177}**	**-9.84×10^{03}**	**2.34×10^{-298}**	**-1.16×10^{02}**	**1.35×10^{-107}**	3	0	**3**
11	RDPSO [25]	**-8.60×10^{02}**	**7.53×10^{-95}**	**-5.09×10^{03}**	**4.92×10^{-270}**	**-7.05×10^{01}**	**1.92×10^{-86}**	3	0	**3**
12	λ-logic [43]	**-3.11×10^{02}**	**6.62×10^{-150}**	**-3.24×10^{03}**	**1.39×10^{-250}**	**-8.02×10^{01}**	**6.29×10^{-92}**	3	0	**3**
13	GPSO	**-7.18×10^{02}**	**8.08×10^{-186}**	**-3.37×10^{04}**	**0.0**	**-8.95×10^{03}**	**2.69×10^{-294}**	3	0	**3**
14	CPSO [29]	**-2.74×10^{02}**	**2.01×10^{-144}**	**-2.53×10^{04}**	**0.0**	-	-	2	0	**2**
15	SPPO [46]	**-3.21×10^{02}**	**2.97×10^{-151}**	**-3.18×10^{03}**	**7.17×10^{-250}**	-	-	2	0	**2**
16	SA-PSO [26]	**-1.79×10^{02}**	**2.80×10^{-126}**	**-4.53×10^{03}**	**4.47×10^{-265}**	-	-	2	0	**2**
17	MIQCQP [41]	**6.12×10^{00}**	**1.82×10^{-08}**	-	-	**-3.45×10^{01}**	**4.49×10^{-57}**	2	0	**2**
18	MSAF [22]	-	-	**-6.52×10^{03}**	**1.06×10^{-280}**	-	-	1	0	**1**
19	EGSSOA [42]	-	-	**-8.04×10^{02}**	**1.07×10^{-190}**	-	-	1	0	**1**
20	SOH-PSO [3]	**-2.55×10^{03}**	**1.94×10^{-240}**	-	-	-	-	1	0	**1**
21	NPSO-LRS [27]	**-3.44×10^{02}**	**2.66×10^{-154}**	-	-	-	-	1	0	**1**
22	QMPSO [30]	**-1.36×10^{03}**	**1.44×10^{-144}**	-	-	-	-	1	0	**1**
23	FDA [38]	**-3.02×10^{02}**	**1.02×10^{-148}**	-	-	-	-	1	0	**1**
24	GA-API [40]	**-3.14×10^{02}**	**2.70×10^{-150}**	-	-	-	-	1	0	**1**